GIÁO TRÌNH

Lập trình Sd & Sd++

Từ căn bản đến nâng cao

Cú pháp phẳng · Khung `sta` và định danh `iid` · Chốt `ed`

Bộ cờ chốt · Con trỏ · Reback · Siêu lập trình

Bản chuẩn Sd 2 / Sd++ 2

Có bài tập, đáp án, bảng tra nhanh và ngữ pháp hình thức

Tác giả ngôn ngữ: **Lương**
Biên soạn giáo trình: Claude (theo đặc tả của tác giả)

Phiên bản 2.0 · tháng 11 năm 2026

Thông tin tài liệu

Tên	Giáo trình lập trình Sd & Sd++ (từ căn bản đến nâng cao)
Phiên bản	2.0, tháng 11 năm 2026
Phạm vi	Ngôn ngữ Sd 2 (phần lõi, Phần I) và Sd++ 2 (chế độ hệ thống, Phần II)
Nguồn đặc tả	Mã Sd/Sd++ do tác giả cung cấp (bản `ssta` / `$` / `₫` / `€` và bản `sta(main)` / `ed` chụp từ màn hình)
Khổ giấy	A4, in hai mặt, lề trong 3,2 cm để đóng gáy hoặc đục lỗ
Màu sắc	Chỉ dùng đen và các mức xám; in trắng đen vẫn đọc đủ
Font	Latin Modern, Liberation Mono; đã nhúng toàn bộ vào tệp PDF

Trạng thái của đặc tả

Sd/Sd++ chưa có trình biên dịch chính thức. Để kiểm chứng sách, người biên soạn đã viết **bộ thông dịch tham chiếu** và **Sd IDE** (Phụ lục G) chạy được cả Sd lẫn Sd++. Toàn bộ 53 chương trình Sd đầy đủ và 11 chương trình Sd++ đầy đủ trong sách đã được chạy, kết quả khớp với kết quả ghi trong sách.

Cách đọc ký hiệu [ĐX]

Cú pháp nào có sẵn trong mã của tác giả được coi là **chính thức** và không đánh dấu. Phần do người biên soạn thêm để ngôn ngữ nhất quán và đủ dùng được ghi chú bằng ký hiệu **[ĐX]** (đề xuất). Phần **[ĐX]** chỉ trở thành chính thức khi tác giả duyệt; tác giả có thể bỏ hoặc đổi tên bất kỳ mục nào mà không làm hỏng phần còn lại của sách. Phụ lục D liệt kê toàn bộ thay đổi so với mã gốc.

Mục lục

Lời nói đầu **ix**

I Sd căn bản **1**

1 Làm quen với Sd **3**
1.1 Sd là gì? . . . 3
1.2 Chương trình đầu tiên . . . 3
1.3 Giải phẫu một câu lệnh . . . 4
1.4 Ba loại thành phần . . . 4
1.5 Ghi chú . . . 5
1.6 Quy ước tệp và trình bày . . . 5

2 Biến, kiểu dữ liệu và toán tử **7**
2.1 Biến là gì? . . . 7
2.2 Các kiểu dữ liệu cơ bản . . . 7
2.3 Quy tắc đặt tên . . . 7
2.4 Phép toán . . . 8
2.5 Phạm vi của biến . . . 9

3 Nhập, xuất và định dạng **11**
3.1 Lệnh in `pri` . . . 11
3.2 In định dạng siêu tốc . . . 11
3.3 Lệnh nhập `ipt` . . . 11
3.4 Chương trình mẫu: đổi nhiệt độ . . . 12

4 Rẽ nhánh: khối `bolean` **13**
4.1 Khi nào cần rẽ nhánh? . . . 13
4.2 Biến `bool` và toán tử logic . . . 14
4.3 Khối lồng nhau . . . 14

5 Vòng lặp: `for` và `while` **17**
5.1 Hai họ vòng lặp, một chốt `dbl` . . . 17
5.2 Vòng `for`: biết trước số lần lặp . . . 17
5.3 Vòng `while`: lặp theo điều kiện . . . 18
5.4 Dừng sớm và bỏ qua: `brk`, `nxt` . . . 18
5.5 Vòng lặp lồng nhau . . . 19

6 Mảng và chuỗi **21**
6.1 Mảng: nhiều giá trị cùng kiểu . . . 21
6.2 Chuỗi . . . 22

7 Hàm và thủ tục: `fn` và `void` 25
7.1 Vì sao chia nhỏ chương trình? . . . 25
7.2 Hàm có kết quả: `fn` . . . 25
7.3 Thủ tục: `void` . . . 26

8 Cấu trúc dữ liệu: `struct` 29
8.1 Gom nhiều thông tin thành một . . . 29
8.2 Mảng các cấu trúc . . . 29
8.3 Cấu trúc và hàm . . . 30

9 Điểm lỗi và gỡ lỗi 33
9.1 Ba loại sự cố . . . 33
9.2 Điểm lỗi . . . 33
9.3 Đọc thông báo lỗi . . . 34
9.4 Thực hành: tìm lỗi trong chương trình sau . . . 34
9.5 Quy trình gỡ lỗi năm bước . . . 34

II Sd++ nâng cao 37

10 Sd++ và ranh giới `ssta` / `##` 39
10.1 Sd++ là gì? . . . 39
10.2 Chiếc hộp bảo mật . . . 39
10.3 Bộ cờ chốt . . . 39
10.4 Ghi chú trong Sd++ . . . 40
10.5 Chương trình Sd++ đầu tiên . . . 40

11 Biến toàn cục trần và khung `define` 43
11.1 Biến toàn cục trần . . . 43
11.2 Khung `define[...]{ ... } €` . . . 43
11.3 Đối chiếu dạng phẳng của Sd++ với Sd . . . 44

12 Con trỏ và bộ nhớ Heap 47
12.1 Con trỏ là gì? . . . 47
12.2 Bộ nhớ Heap . . . 47
12.3 Quy tắc an toàn khi dùng bộ nhớ . . . 48

13 Khối chọn lọc `ids sb` và Reback 51
13.1 Ý tưởng: tự kiểm tra lại . . . 51
13.2 Cú pháp . . . 51
13.3 Sơ đồ hoạt động . . . 52
13.4 Ví dụ: hệ thống tự sưởi . . . 52
13.5 Quy tắc chống vòng vô hạn . . . 53
13.6 Đối chiếu với `while` . . . 53

14 Siêu lập trình: `defk` và cờ chốt tùy biến **55**
14.1 Tự tạo cờ chốt . . . 55
14.2 Đọc từng phần của `defk` . . . 55
14.3 Quy tắc của `defk` . . . 56
14.4 Vùng tự chốt `:&` . . . 56

15 Thư viện chuẩn **59**
15.1 Toán học . . . 59
15.2 Tệp . . . 59
15.3 Dự án nhiều tệp[ĐX] . . . 60

16 Trình biên dịch Sd và Sd++ **61**
16.1 Dây chuyền . . . 61
16.2 Phân tích từ vựng . . . 61
16.3 Phân tích cú pháp . . . 61
16.4 Kiểm tra ngữ nghĩa . . . 61
16.5 Phục hồi sau lỗi . . . 62
16.6 Lộ trình hiện thực . . . 62

17 Dự án tổng hợp **63**
17.1 Dự án 1: Sổ điểm lớp (Sd) . . . 63
17.2 Dự án 2: Giám sát máy chủ tự chữa lành (Sd++) . . . 64

A Bảng từ khóa và ký hiệu **67**

B Ngữ pháp hình thức (EBNF) **69**

C Bảng mã lỗi và cảnh báo **71**

D Đối chiếu bản gốc và Sd 2 **73**
D.1 Từ bản chụp màn hình sang Sd 2 . . . 73
D.2 Từ đặc tả Sd++ cũ sang Sd++ 2 . . . 73

E Bảng tra nhanh **75**

F Đáp án bài tập **77**

G Sd IDE, bộ thông dịch và kết quả kiểm chứng **89**

H Thuật ngữ **91**

Lời nói đầu

Sd là ngôn ngữ lập trình theo triết lý *phẳng*: mọi câu lệnh đều có cùng một dáng, một khung `sta`, một định danh `iid` và một chốt `ed`. Người đọc không phải nhớ hàng chục kiểu ngoặc và dấu chấm phẩy khác nhau; chỉ cần nhớ một khuôn và biết khuôn đó đổi hình thế nào cho từng việc: khai báo, in, nhập, lặp, rẽ nhánh, hàm.

Sd++ là phần mở rộng cho lập trình hệ thống: bộ cờ chốt `$`, `₫`, `€`, `$$`, `#`, `##`, con trỏ, bộ nhớ Heap, khối chọn lọc tự kiểm tra lại (Reback) và siêu lập trình `defk`. Mọi chương trình Sd đều là chương trình Sd++ hợp lệ khi đặt trong khung `ssta { ... }`.

Cuốn sách này làm ba việc.

1. **Chốt một cú pháp nhất quán.** Bản mã chụp từ màn hình có nhiều chỗ tự mâu thuẫn, chẳng hạn `==` vừa nối đặc tả vừa so sánh, vòng `for` thiếu khởi tạo, hai trường cùng tên `a` trong một cấu trúc. Sách giữ nguyên tinh thần và từ khóa của tác giả nhưng sửa các điểm đó (Phụ lục D).
2. **Dạy từ đầu đến nâng cao.** Phần I (chương 1–9) đi từ chương trình đầu tiên đến hàm, mảng, cấu trúc và gỡ lỗi. Phần II (chương 10–17) dạy Sd++ và kết thúc bằng hai dự án hoàn chỉnh.
3. **Làm tài liệu bền để in.** Có bài tập sau mỗi chương, đáp án ở Phụ lục F, bảng tra nhanh một trang ở Phụ lục E và ngữ pháp hình thức ở Phụ lục B, để cuốn sách vẫn dùng được làm tài liệu tra cứu sau khi đã học xong.

Lộ trình học gợi ý (8 tuần)

Tuần	Chương	Mục tiêu cuối tuần
1	1, 2	Viết, đọc được chương trình có biến và phép tính
2	3, 4	Nhập dữ liệu, rẽ nhánh nhiều trường hợp
3	5	Thành thạo `for`, `while`, `brk`, `nxt`
4	6, 7	Mảng, chuỗi, chia chương trình thành hàm
5	8, 9	Cấu trúc dữ liệu; tự đọc thông báo lỗi và sửa
6	10, 11, 12	Sd++: cờ chốt, biến toàn cục, con trỏ, Heap
7	13, 14, 15	Reback, `defk`, thư viện chuẩn
8	16, 17	Hiểu trình biên dịch; làm trọn một dự án

Cách học

Gõ lại từng ví dụ thay vì chỉ đọc. Làm bài tập trước khi xem đáp án. Khi gặp lỗi, đọc mã lỗi ở Phụ lục C trước, rồi mới hỏi người khác.

Phần I

Sd căn bản

Chương 1

Làm quen với Sd

1.1 Sd là gì?

Sd là ngôn ngữ lập trình *phẳng*. Nhìn vào một chương trình Sd, bạn thấy các câu lệnh xếp thẳng hàng, mỗi lệnh có cùng một khuôn mặt. Có ba thành phần làm nên khuôn mặt đó:

- **`sta`** (Statement Authority), *thể xác*: dựng khung cho lệnh, nơi chứa điều kiện, vòng lặp, hàm và các phạm vi.
- **`iid`** (Instruction Identity), *linh hồn*: cho biết đây là một lệnh cần thực thi (in, nhập, gán, lặp…).
- **`ed`** (end), *chốt*: đánh dấu câu lệnh hoặc khối đã kết thúc.

Thể xác không có linh hồn thì chỉ là cái khung rỗng; linh hồn không có thể xác thì không có chỗ để thực thi. Vì vậy hầu hết lệnh trong Sd bắt đầu bằng `iid:` và chứa một khung `sta(...)`.

1.2 Chương trình đầu tiên

Ví dụ 1.1. Chương trình Hello

```
imp*ioprt.ed

sta(main){
    iid: sta(pri) => "Xin chao, Sd!" ed
    iid: ref* 0 ed
}
```

Kết quả in ra màn hình:

```
Xin chao, Sd!
```

Đọc từng dòng:

Dòng	Ý nghĩa
`imp*ioprt.ed`	Nạp thư viện nhập xuất `ioprt`. Dòng import luôn có dạng `imp*TEN.ed` và đứng ở đầu tệp.
`sta(main){`	Mở khung chương trình chính. Mọi chương trình có đúng một `sta(main)`; chương trình bắt đầu chạy từ đây.
`iid: sta(pri) => "..." ed`	Lệnh in. `pri` là loại lệnh, dấu `=>` nối loại lệnh với nội dung, `ed` chốt câu lệnh.
`iid: ref* 0 ed`	Kết thúc chương trình và trả về mã `0` (thành công). Khung `main` *bắt buộc* khép bằng lệnh `ref*`.
`}`	Đóng khung `main`.

Vì sao `ref*` bắt buộc?

Trong bản gốc, dòng `ref* 0 ed:` chỉ là thói quen kết thúc. Sd 2 biến nó thành quy tắc: chương trình luôn nói rõ mình kết thúc với giá trị nào. Giá trị `0` nghĩa là mọi thứ đúng; giá trị khác `0` là *điểm lỗi* do bạn tự quy định (chương 9).

1.3 Giải phẫu một câu lệnh

Mọi câu lệnh Sd đều có cùng một khuôn:

Khuôn câu lệnh

`iid: sta(ĐẦU) => THẺ: sta(THÂN) ed`

Phần `=> THẺ: sta(THÂN)` chỉ có khi lệnh cần thêm thông tin; lệnh đơn giản như `iid: ref* 0 ed` không cần.

Các phần của lệnh khai báo có giá trị đầu

```
iid: sta(int a) => edt: sta(a = 5) ed
|    |   |        |    |   |        |
|    |   |        |    |   |        +-- ed    : chot cau lenh
|    |   |        |    |   +----------- than  : noi dung (a = 5)
|    |   |        |    +--------------- khung : sta(...) chua than
|    |   |        +-------------------- the   : edt = "chinh sua gia tri"
|    |   +------------------------------ noi   : => noi dau voi phan sau
|    +---------------------------------- dau   : sta(int a) = khai bao bien a
+------------------------------------- iid   : day la mot lenh
```

Hai điều cần nhớ:

1. `=>` chỉ *nối* phần đầu với phần thân. Nó không phải phép so sánh; so sánh dùng `==` (chương 2). In bản gốc, `==` đảm nhiệm cả hai việc, dễ gây nhầm, nên Sd 2 tách ra.
2. Một câu lệnh chỉ có *một* chốt `ed`, ở cuối.

1.4 Ba loại thành phần

Trong một chương trình Sd bạn chỉ gặp ba loại thành phần:

Loại	Nhận dạng	Ví dụ
Lệnh đơn	`iid: ... ed` trên một dòng	`iid: sta(pri) => "Hi" ed`
Khối	`iid: sta(loại) => thẻ:` nhiều dòng, khép bằng `tên-khép ed`	vòng lặp, rẽ nhánh, hàm
Ghi chú	nằm giữa hai cặp `\\`	`\\ day la ghi chu \\`

Khối sẽ được học dần từ chương 4. Để dễ nhớ, mọi khối đều theo khuôn:

Khuôn của một khối

```
iid: sta(LOAI) => THE1,THE2:      <- dau khoi, ket thuc bang dau hai cham
    ...than khoi...
TEN-KHEP ed                       <- khep khoi
```

1.5 Ghi chú

Ghi chú là chữ dành cho người đọc; trình biên dịch bỏ qua. Ghi chú nằm giữa hai cặp dấu gạch ngược `\\`, có thể trên một dòng hoặc nhiều dòng.

Ví dụ 1.2. Ghi chú một dòng và nhiều dòng

```
imp*ioprt.ed

\\ Chuong trinh chao hai dong.
   Ghi chu khoi co the dai nhieu dong. \\
sta(main){
    iid: sta(pri) => "Dong 1" ed      \\ in dong 1 \\
    iid: sta(pri) => "Dong 2" ed
    iid: ref* 0 ed
}
```

Ghi chú phải có cặp đóng

Bản gốc cảnh báo: nếu mở `\\` mà quên đóng, trình đọc sẽ nuốt toàn bộ phần mã còn lại thành ghi chú. Sd 2 không nuốt âm thầm: khi chạm cuối tệp mà ghi chú vẫn mở, trình biên dịch báo lỗi `E101` kèm *số dòng nơi ghi chú được mở*, để bạn tìm ra chỗ quên đóng ngay.

1.6 Quy ước tệp và trình bày

- Tệp mã Sd lưu bằng mã hóa UTF-8, đuôi `.sd`[ĐX].
- Mỗi lệnh đơn một dòng; thụt lề 4 dấu cách cho mỗi cấp khung.
- Chuỗi và ghi chú có thể chứa chữ có dấu. Các ví dụ trong sách viết không dấu cho dễ gõ trên mọi bàn phím.
- Chữ hoa và chữ thường là khác nhau: `Ten` và `ten` là hai tên khác nhau.

Tóm tắt chương

Mọi lệnh có khuôn `iid: sta(...) ... ed`. Dấu `=>` nối đầu với thân. Khối khép bằng `tên-khép ed`. Khung `main` khép bằng `ref*`. Ghi chú nằm giữa hai cặp `\\`.

Bài tập

1.1 Viết chương trình in ra tên của bạn trên một dòng và lớp của bạn trên dòng kế tiếp.

1.2 Thêm vào chương trình trên một ghi chú nhiều dòng ở đầu tệp, ghi ngày viết và mục đích.

1.3 Dòng nào dưới đây *sai* khuôn câu lệnh và sai ở đâu?
`iid: sta(pri) => "A"  iid sta(pri) => "B" ed  iid: sta(pri) => "C" ed`

1.4 Chương trình sau thiếu gì để hợp lệ? `sta(main){ iid: sta(pri) => "x" ed }`

Chương 2

Biến, kiểu dữ liệu và toán tử

2.1 Biến là gì?

Biến là một ô nhớ có tên, chứa một giá trị thuộc một kiểu nhất định. Khai báo biến trong Sd dùng khung `sta(kiểu tên)`:

Ví dụ 2.1. Khai báo biến

```
sta(main){
    iid: sta(int a) ed                          \\ khai bao, mac dinh a = 0 \\
    iid: sta(int b) => edt: sta(b = 7) ed       \\ khai bao va dat gia tri \\
    iid: ref* 0 ed
}
```

Có hai dạng:

1. `iid: sta(int a) ed`: chỉ khai báo.
2. `iid: sta(int b) => edt: sta(b = 7) ed`: khai báo *và* tạo luôn giá trị. Thẻ `edt` (edit) báo rằng phần thân *sửa* giá trị của biến.

Giá trị mặc định[ĐX]

Biến khai báo mà không gán giá trị không bao giờ ở trạng thái "rác": `int` bằng `0`, `float` bằng `0.0`, `str` bằng chuỗi rỗng `""`, `bool` bằng `false`. Nhờ vậy cùng một chương trình luôn cho cùng một kết quả.

2.2 Các kiểu dữ liệu cơ bản

Kiểu	Ý nghĩa	Giá trị mẫu	Mặc định
`int`	số nguyên	`42`, `-7`	`0`
`float`	số thực	`3.14`, `-0.5`	`0.0`
`str`	chuỗi ký tự	`"Sd++"`	`""`
`bool`	đúng hoặc sai	`true`, `false`	`false`

Ngoài ra Sd có kiểu mảng (chương 6), kiểu cấu trúc (chương 8); Sd++ có thêm kiểu con trỏ `ptr` (chương 12).

2.3 Quy tắc đặt tên

- Tên gồm chữ cái không dấu, chữ số và dấu gạch dưới; không bắt đầu bằng chữ số.
- Không trùng từ khóa (danh sách ở Phụ lục A).
- Trong cùng một khung `{ ... }` không được khai báo hai biến cùng tên (lỗi `E302`).
- Tên nên nói lên ý nghĩa: `tong_tien` tốt hơn `tt`.

2.4 Phép toán

Nhóm	Toán tử	Ghi chú
Số học	`+ - * / %`	`/` trên hai số nguyên là chia nguyên; `%` lấy dư
Tăng/giảm	`i++  i--`	chỉ dùng như một câu lệnh hoặc trong đầu vòng lặp
So sánh	`== != < <= > >=`	cho kết quả `bool`
Logic[ĐX]	`and  or  not`	kết hợp các giá trị `bool`
Gán	`=`	chỉ xuất hiện trong thân của lệnh

Thứ tự ưu tiên, từ cao xuống thấp (ngang hàng thì tính từ trái sang phải):

Bậc	Toán tử
1	`( ) [ ] .`
2	`not` `-` (dấu âm)
3	`*  /  %`
4	`+  -`
5	`<  <=  >  >=`
6	`==  !=`
7	`and`
8	`or`

Ví dụ 2.2. Chia nguyên, chia dư, chia số thực

```
imp*ioprt.ed

sta(main){
    iid: sta(int a) => edt: sta(a = 7) ed
    iid: sta(int b) => edt: sta(b = 2) ed
    iid: sta(int q) => edt: sta(q = a / b) ed     \\ q = 3 \\
    iid: sta(int r) => edt: sta(r = a % b) ed     \\ r = 1 \\
    iid: sta(float f) => edt: sta(f = 7.0 / 2.0) ed
    iid: sta(pri) => "q=/d{q} r=/d{r} f=/f{f}" ed
    iid: ref* 0 ed
}
```

Kết quả theo đặc tả:

```
q=3 r=1 f=3.50
```

Gán là =, so sánh là ==

Trong bản gốc, một ô điều kiện được viết `sta(a = 0)`, dùng dấu gán ở chỗ cần so sánh. Điều đó sẽ gán `0` cho `a` thay vì hỏi `a` có bằng `0` hay không. Sd 2 quy định rõ: `=` chỉ gán, `==` chỉ so sánh, và dùng `=` trong một điều kiện là lỗi `E303`.

2.5 Phạm vi của biến

Biến sống trong khung `{ ... }` gần nhất chứa nó và biến mất khi khung đóng. Biến khai báo trong đầu vòng lặp `for` chỉ sống trong vòng lặp đó. Dùng biến ngoài phạm vi là lỗi `E301` (biến chưa khai báo).

Tóm tắt chương

Khai báo: `iid: sta(kiểu tên) ed`; có giá trị đầu: thêm `=> edt: sta(tên = giá trị)`. Bốn kiểu cơ bản: `int`, `float`, `str`, `bool`. `=` gán, `==` so sánh. Chia hai số nguyên cho kết quả nguyên.

Bài tập

2.1 Tính giá trị: `17 / 5`, `17 % 5`, `17 / 5.0`, `2 + 3 * 4`, `(2 + 3) * 4`.

2.2 Viết chương trình khai báo hai số nguyên `a = 12`, `b = 5` rồi in tổng, hiệu, tích, thương nguyên và số dư.

2.3 Viết chương trình tính chu vi và diện tích hình tròn bán kính `r = 2.5`, dùng `pi = 3.14159`.

2.4 Giải thích vì sao `sta(a = 0)` bị cấm làm điều kiện trong Sd 2.

Chương 3

Nhập, xuất và định dạng

3.1 Lệnh in `pri`

Lệnh in có dạng `iid: sta(pri) => BIỂU-THỨC ed`. Mỗi lệnh `pri` in xong tự xuống dòng. In chuỗi rỗng `""` sẽ in một dòng trống.

3.2 In định dạng siêu tốc

Thay vì nối chuỗi bằng dấu cộng, Sd nhúng giá trị thẳng vào trong chuỗi bằng *chỗ trống định dạng*:

Chỗ trống	Nhúng	Ví dụ
`/s{x}`	chuỗi `str`	`/s{ten}`
`/d{x}`	số nguyên `int`	`/d{a + b}`
`/f{x}`	số thực, 2 chữ số lẻ	`/f{diem}`
`/f3{x}`	số thực, 3 chữ số lẻ[ĐX]	`/f3{pi}`
`/b{x}`[ĐX]	giá trị `bool`	`/b{xong}`
`//`	một dấu `/` thật	`"1//2"`

Trong cặp `{ }` có thể đặt tên biến, phần tử mảng, trường của cấu trúc hoặc cả một biểu thức.

Dấu ngoặc nhọn là điểm nâng cấp[ĐX]

Bản gốc viết liền, ví dụ `/dtemp`. Cách đó nhập nhằng khi tên biến dính với chữ đứng sau: `/dtempC` là biến `tempC` hay `temp` theo sau chữ `C`? Cặp `{ }` chỉ rõ ranh giới. Trình biên dịch Sd++ vẫn đọc được dạng viết liền cũ nhưng cảnh báo.

3.3 Lệnh nhập `ipt`

Nhập giá trị từ bàn phím cho một biến có sẵn: `iid: sta(ipt) => edt: sta(tên) ed`. Kiểu dữ liệu đọc vào lấy theo kiểu của biến.

Ví dụ 3.1. Nhập hai số và in tổng

```
imp*ioprt.ed

sta(main){
    iid: sta(int a) ed
    iid: sta(int b) ed
    iid: sta(pri) => "Nhap a:" ed
    iid: sta(ipt) => edt: sta(a) ed
    iid: sta(pri) => "Nhap b:" ed
    iid: sta(ipt) => edt: sta(b) ed
    iid: sta(pri) => "/d{a} + /d{b} = /d{a + b}" ed
    iid: ref* 0 ed
}
```

Nếu người dùng gõ không đúng kiểu (ví dụ chữ cho biến `int`), chương trình dừng với lỗi `E403`[ĐX]. Chương 9 chỉ cách kiểm soát tình huống đó.

3.4 Chương trình mẫu: đổi nhiệt độ

Ví dụ 3.2. Đổi độ C sang độ F

```
imp*ioprt.ed

sta(main){
    iid: sta(float c) ed
    iid: sta(float f) ed
    iid: sta(pri) => "Nhap do C:" ed
    iid: sta(ipt) => edt: sta(c) ed
    iid: sta(f = c * 9.0 / 5.0 + 32.0) ed
    iid: sta(pri) => "/f{c} do C = /f{f} do F" ed
    iid: ref* 0 ed
}
```

Dòng `iid: sta(f = c * 9.0 / 5.0 + 32.0) ed` là **lệnh gán rút gọn**[ĐX]: khi đầu lệnh là một phép gán vào biến đã có, không cần viết `=> edt:` dài dòng. Hai cách viết sau tương đương:

```
iid: sta(f) => edt: sta(f = c * 9.0 / 5.0 + 32.0) ed       \\ day du \\
iid: sta(f = c * 9.0 / 5.0 + 32.0) ed                      \\ rut gon \\
```

Tóm tắt chương

`pri` in và xuống dòng; `ipt` nhập theo kiểu của biến. Chỗ trống `/s{ }`, `/d{ }`, `/f{ }`, `/b{ }` nhúng giá trị vào chuỗi, `//` in dấu `/`.

Bài tập

3.1 Viết chương trình nhập tên và tuổi rồi in: `Xin chao An, nam sau ban 20 tuoi`, trong đó 20 là tuổi cộng thêm 1.

3.2 Nhập hai số thực là chiều dài và chiều rộng, in chu vi và diện tích hình chữ nhật với hai chữ số lẻ.

3.3 Nhập một số nguyên `n` có ba chữ số; in chữ số hàng trăm, hàng chục, hàng đơn vị (dùng `/` và `%`).

3.4 In ra đúng dòng: `Ti le: 3/4`. Viết lệnh `pri` phù hợp.

Chương 4

Rẽ nhánh: khối `bolean`

4.1 Khi nào cần rẽ nhánh?

Chương trình thường phải quyết định: điểm đủ cao thì đỗ, mật khẩu đúng thì cho vào. Sd làm việc này bằng khối `bolean`, "cấu trúc logic đúng–sai để kiểm tra điều kiện". Khối có hai thẻ đặc tả: `cdn` (condition, điều kiện) và `tf` (true–false, hai nhánh đúng hoặc sai); khép bằng `bln ed`.

Khuôn khối `bolean`

```
iid: sta(bolean) => cdn,tf:
    sta(DIEU-KIEN-1) {
        ...
    }
    sta(DIEU-KIEN-2) {          \\ co the co nhieu nhanh \\
        ...
    }
    sta(else) {                 \\ tuy chon: khi khong dieu kien nao dung \\
        ...
    }
bln ed
```

Các nhánh được xét **từ trên xuống dưới**; nhánh đầu tiên có điều kiện đúng được chạy, các nhánh sau bị bỏ qua. Nhánh `sta(else)`[ĐX]nếu có phải nằm cuối (lỗi `E306` nếu không).

Ví dụ 4.1. Xếp loại theo điểm

```
1  imp*ioprt.ed
2
3  sta(main){
4      iid: sta(float diem) ed
5      iid: sta(pri) => "Nhap diem:" ed
6      iid: sta(ipt) => edt: sta(diem) ed
7      iid: sta(bolean) => cdn,tf:
8          sta(diem >= 8.0) {
9              iid: sta(pri) => "Gioi" ed
10         }
11         sta(diem >= 6.5) {
12             iid: sta(pri) => "Kha" ed
13         }
14         sta(diem >= 5.0) {
15             iid: sta(pri) => "Trung binh" ed
16         }
17         sta(else) {
18             iid: sta(pri) => "Yeu" ed
19         }
20     bln ed
21     iid: ref* 0 ed
22 }
```

Với `diem = 7.0`, nhánh đầu sai, nhánh thứ hai đúng nên in `Kha`; hai nhánh còn lại không được xét.

4.2 Biến `bool` và toán tử logic

Kết quả của một phép so sánh là giá trị `bool`. Bạn có thể lưu nó vào biến rồi dùng lại:

Ví dụ 4.2. Biến bool và `and`, `or`, `not`

```
imp*ioprt.ed

sta(main){
    iid: sta(int tuoi) => edt: sta(tuoi = 17) ed
    iid: sta(bool nguoi_lon) => edt: sta(nguoi_lon = tuoi >= 18) ed
    iid: sta(bool hoc_sinh) => edt: sta(hoc_sinh = true) ed
    iid: sta(bool giam_gia) => edt: sta(giam_gia = hoc_sinh or tuoi < 12) ed
    iid: sta(pri) => "nguoi lon: /b{nguoi_lon}" ed
    iid: sta(pri) => "giam gia : /b{giam_gia}" ed
    iid: ref* 0 ed
}
```

4.3 Khối lồng nhau

Trong một nhánh có thể đặt khối `bolean` khác. Mỗi khối tự khép bằng `bln ed` của chính nó.

Ví dụ 4.3. Số chẵn, lẻ, âm, dương

```
imp*ioprt.ed

sta(main){
    iid: sta(int n) => edt: sta(n = -4) ed
    iid: sta(bolean) => cdn,tf:
        sta(n > 0) {
            iid: sta(pri) => "duong" ed
        }
        sta(n < 0) {
            iid: sta(pri) => "am" ed
            iid: sta(bolean) => cdn,tf:
                sta(n % 2 == 0) {
                    iid: sta(pri) => "va la so chan" ed
                }
            bln ed
        }
        sta(else) {
            iid: sta(pri) => "bang khong" ed
        }
    bln ed
    iid: ref* 0 ed
}
```

Lỗi thường gặp

- Quên `bln ed` ở cuối khối: lỗi `E202` (khối chưa khép).
- Khép sai loại, ví dụ `dbl ed` thay cho `bln ed`: lỗi `E202`.
- Đặt các nhánh theo thứ tự sai, chẳng hạn xét `diem >= 5.0` trước `diem >= 8.0`: điểm 9 sẽ rơi vào nhánh "trung bình" vì nhánh đó đúng trước. Luôn xét điều kiện *hẹp* trước, *rộng* sau.

Tóm tắt chương

Rẽ nhánh dùng `iid: sta(bolean) => cdn,tf:`, các nhánh `sta(điều kiện) { ... }`, nhánh cuối `sta(else)` tùy chọn, khép bằng `bln ed`. Chỉ một nhánh chạy: nhánh đúng đầu tiên từ trên xuống.

Bài tập

4.1 Nhập một số nguyên, in `chan` hoặc `le`.

4.2 Nhập ba số nguyên, in số lớn nhất.

4.3 Nhập năm, in `nhuan` nếu chia hết cho 400, hoặc chia hết cho 4 mà không chia hết cho 100; ngược lại in `khong nhuan`.

4.4 Nhập hệ số `a`, `b` của phương trình `a*x + b = 0`; in nghiệm, hoặc `vo nghiem`, hoặc `vo so nghiem` (chú ý các trường hợp `a = 0`).

Chương 5

Vòng lặp: `for` và `while`

5.1 Hai họ vòng lặp, một chốt `dbl`

Vòng lặp thuộc *họ đôi*: khối có **hai thẻ đặc tả** và khép bằng `dbl ed` (double). Chữ `dbl` đứng ở hai nơi: mở thân vòng (`dbl sta(...) { ... }`) và khép khối (`dbl ed`).

Vòng lặp	Hai thẻ	Ý nghĩa của hai thẻ
`for`	`cdn,chck`	điều kiện dừng (`cdn`), kiểm soát bước nhảy (`chck`)
`while`	`cdn,edt`	điều kiện (`cdn`), thân chỉnh biến để điều kiện đổi (`edt`)

5.2 Vòng `for`: biết trước số lần lặp

Khuôn vòng for

```
iid: sta(for) => cdn,chck:
    dbl sta(KHOI-TAO ' DIEU-KIEN ' BUOC) {
        ...than vong lap...
    }
dbl ed
```

Phần trong `sta( )` có ba mục ngăn cách bằng dấu nháy đơn `'`: khởi tạo, điều kiện, bước. Biến khai báo ở mục khởi tạo chỉ sống trong vòng lặp. Mục bước có thể là `i++`, `i--` hoặc một phép gán như `i = i + 2`. Thiếu một trong ba mục là lỗi `E206`.

Ví dụ 5.1. Tổng từ 1 đến 10

```
1  imp*ioprt.ed
2
3  sta(main){
4      iid: sta(int tong) => edt: sta(tong = 0) ed
5      iid: sta(for) => cdn,chck:
6          dbl sta(int i = 1 ' i <= 10 ' i++) {
7              iid: sta(tong = tong + i) ed
8          }
9      dbl ed
10     iid: sta(pri) => "Tong = /d{tong}" ed
11     iid: ref* 0 ed
12 }
```

Kết quả: `Tong = 55`.

Vì sao đầu vòng `for` phải đủ ba mục?

Bản gốc viết `sta(i ' i < 10 ' i++)` mà không cho biết `i` bắt đầu từ đâu. Nếu `i` chưa được khai báo, trình biên dịch không đoán được; nếu đã khai báo thì lần chạy thứ hai sẽ bắt đầu từ giá trị cũ. Ba mục đầy đủ làm vòng lặp luôn cho cùng một kết quả mỗi lần chạy.

5.3 Vòng `while`: lặp theo điều kiện

Khuôn vòng while

```
iid: sta(while) => cdn,edt:
    dbl sta(DIEU-KIEN) {
        ...than: PHAI chinh it nhat mot bien co trong DIEU-KIEN...
    }
dbl ed
```

Thẻ `edt` nhắc một quy tắc logic quan trọng: nếu thân vòng `while` không bao giờ sửa biến nào trong điều kiện thì điều kiện không bao giờ đổi, vòng lặp sẽ chạy mãi. Trình biên dịch Sd 2 phát hiện điều này và báo cảnh báo `W201`[ĐX].

Ví dụ 5.2. Tổng các chữ số của một số

```
imp*ioprt.ed

sta(main){
    iid: sta(int n) => edt: sta(n = 12345) ed
    iid: sta(int tong) => edt: sta(tong = 0) ed
    iid: sta(while) => cdn,edt:
        dbl sta(n > 0) {
            iid: sta(tong = tong + n % 10) ed
            iid: sta(n = n / 10) ed
        }
    dbl ed
    iid: sta(pri) => "Tong cac chu so = /d{tong}" ed
    iid: ref* 0 ed
}
```

Kết quả: `Tong cac chu so = 15`.

5.4 Dừng sớm và bỏ qua: `brk`, `nxt`

- `iid: brk ed`[ĐX] thoát hẳn khỏi vòng lặp gần nhất.
- `iid: nxt ed`[ĐX] bỏ qua phần còn lại của lượt này, sang lượt kế tiếp.

Ví dụ 5.3. Bội số đầu tiên của 7 lớn hơn 100

```
imp*ioprt.ed

sta(main){
    iid: sta(for) => cdn,chck:
        dbl sta(int i = 101 ' i <= 200 ' i++) {
            iid: sta(bolean) => cdn,tf:
                sta(i % 7 == 0) {
                    iid: sta(pri) => "Boi 7 dau tien: /d{i}" ed
                    iid: brk ed
                }
            bln ed
        }
    dbl ed
    iid: ref* 0 ed
}
```

Kết quả: `Boi 7 dau tien: 105`.

5.5 Vòng lặp lồng nhau

Ví dụ 5.4. Bảng cửu chương 2 đến 4

```
imp*ioprt.ed

sta(main){
    iid: sta(for) => cdn,chck:
        dbl sta(int a = 2 ' a <= 4 ' a++) {
            iid: sta(for) => cdn,chck:
                dbl sta(int b = 1 ' b <= 3 ' b++) {
                    iid: sta(pri) => "/d{a} x /d{b} = /d{a * b}" ed
                }
            dbl ed
        }
    dbl ed
    iid: ref* 0 ed
}
```

Mỗi vòng `for` tự khép bằng `dbl ed` của mình; vòng trong khép trước vòng ngoài.

Lỗi thường gặp

- Vòng `while` không chỉnh biến điều kiện (cảnh báo `W201`, chạy vô hạn).
- Lệch một đơn vị: viết `i < 10` khi cần `i <= 10`.
- Khép nhầm: vòng trong khép `dbl ed` thiếu một lần, vòng ngoài bị "ăn" mất chốt.

Tóm tắt chương

Vòng lặp là khối họ đôi, khép bằng `dbl ed`. `for` có ba mục ngăn bởi `'`, đủ khởi tạo, điều kiện, bước. `while` có một điều kiện và thân phải chỉnh biến trong điều kiện. `brk` thoát, `nxt` bỏ qua lượt.

Bài tập

5.1 In các số chẵn từ 2 đến 20, mỗi số một dòng.

5.2 Nhập `n`, tính `n!` (tích từ 1 đến `n`) bằng vòng `for`.

5.3 Nhập `n`, đếm số chữ số của `n` bằng vòng `while`.

5.4 In hình tam giác sao cao 4 dòng (dòng `k` có `k` dấu `*`). Gợi ý: dùng hai vòng `for` lồng nhau và biến chuỗi `str` ghép bằng thư viện `str` (chương 6).

5.5 Tìm và sửa lỗi: một vòng `while` có điều kiện `n != 0` nhưng thân chỉ chứa lệnh in.

Chương 6

Mảng và chuỗi

6.1 Mảng: nhiều giá trị cùng kiểu

Mảng là dãy các ô nhớ liền nhau, cùng kiểu, đánh số bắt đầu từ `0`. Khai báo kích thước ngay trong ngoặc vuông; các phần tử trong danh sách giá trị ngăn cách bằng dấu nháy đơn `'` (cùng quy tắc ngăn cách như đầu vòng `for`).

Ví dụ 6.1. Khai báo mảng, đọc và ghi phần tử

```
imp*ioprt.ed

sta(main){
    iid: sta(int a[5]) => edt: sta(a = [4 ' 9 ' 1 ' 7 ' 3]) ed
    iid: sta(a[0] = 10) ed
    iid: sta(pri) => "a[0] = /d{a[0]}, a[4] = /d{a[4]}" ed
    iid: ref* 0 ed
}
```

Dấu phẩy và dấu nháy đơn

Sd dùng hai dấu ngăn cách với hai việc riêng biệt, không bao giờ lẫn:

- **Dấu phẩy `,`** chỉ ngăn các *thẻ đặc tả* ở đầu khối: `cdn,chck`.
- **Dấu nháy đơn `'`** ngăn các *mục dữ liệu*: phần tử mảng, tham số hàm, mục của đầu `for`, trường của cấu trúc.

Chỉ số hợp lệ là từ `0` đến kích thước trừ một. Truy cập ngoài khoảng đó khi chạy gây lỗi `E404`.

6.1.1 Duyệt mảng, tìm lớn nhất

Ví dụ 6.2. Tìm phần tử lớn nhất

```
imp*ioprt.ed

sta(main){
    iid: sta(int a[5]) => edt: sta(a = [4 ' 9 ' 1 ' 7 ' 3]) ed
    iid: sta(int mx) => edt: sta(mx = a[0]) ed
    iid: sta(for) => cdn,chck:
        dbl sta(int i = 1 ' i < 5 ' i++) {
            iid: sta(bolean) => cdn,tf:
                sta(a[i] > mx) {
                    iid: sta(mx = a[i]) ed
                }
            bln ed
        }
    dbl ed
    iid: sta(pri) => "Max = /d{mx}" ed
    iid: ref* 0 ed
}
```

Kết quả: `Max = 9`.

6.1.2 Sắp xếp nổi bọt

Ví dụ 6.3. Sắp xếp tăng dần

```
imp*ioprt.ed

sta(main){
    iid: sta(int a[5]) => edt: sta(a = [5 ' 2 ' 9 ' 1 ' 7]) ed
    iid: sta(for) => cdn,chck:
        dbl sta(int i = 0 ' i < 4 ' i++) {
            iid: sta(for) => cdn,chck:
                dbl sta(int j = 0 ' j < 4 - i ' j++) {
                    iid: sta(bolean) => cdn,tf:
                        sta(a[j] > a[j + 1]) {
                            iid: sta(int t) => edt: sta(t = a[j]) ed
                            iid: sta(a[j] = a[j + 1]) ed
                            iid: sta(a[j + 1] = t) ed
                        }
                    bln ed
                }
            dbl ed
        }
    dbl ed
    iid: sta(for) => cdn,chck:
        dbl sta(int k = 0 ' k < 5 ' k++) {
            iid: sta(pri) => "a[/d{k}] = /d{a[k]}" ed
        }
    dbl ed
    iid: ref* 0 ed
}
```

Kết quả: `a[0] = 1`, `a[1] = 2`, `a[2] = 5`, `a[3] = 7`, `a[4] = 9`, mỗi giá trị một dòng.

6.2 Chuỗi

Chuỗi `str` là dãy ký tự đặt trong cặp nháy kép. Các thao tác chuỗi nằm trong thư viện `str` (nạp bằng `imp*str.ed`):

Hàm	Kết quả	Ví dụ
`str.len(s)`	độ dài (số ký tự)	`str.len("Sd++")` → `4`
`str.upper(s)`	chữ hoa	`str.upper("sd")` → `"SD"`
`str.lower(s)`	chữ thường	`str.lower("SD")` → `"sd"`
`str.concat(a ' b)`	ghép hai chuỗi	`str.concat("Sd" ' "++")` → `"Sd++"`

Hai chuỗi được so sánh bằng `==` và `!=`.

Ví dụ 6.4. Các thao tác chuỗi

```
imp*ioprt.ed
imp*str.ed

sta(main){
    iid: sta(str ten) => edt: sta(ten = "Sd++") ed
    iid: sta(int n) => edt: sta(n = str.len(ten)) ed
    iid: sta(str hoa) => edt: sta(hoa = str.upper("sd lang")) ed
```

```
    iid: sta(str ghep) => edt: sta(ghep = str.concat(ten ' "!")) ed
    iid: sta(pri) => "/s{ten} dai /d{n} ky tu" ed
    iid: sta(pri) => "/s{hoa} va /s{ghep}" ed
    iid: ref* 0 ed
}
```

Kết quả:

```
Sd++ dai 4 ky tu
SD LANG va Sd++!
```

Tóm tắt chương

Mảng: `sta(kiểu tên[N])`, chỉ số từ 0, giá trị đầu `[x ' y ' z]`. Chuỗi: `str`, thao tác qua thư viện `str`. Dấu `,` ngăn thẻ, dấu `'` ngăn dữ liệu.

Bài tập

6.1 Cho mảng `[3 ' 8 ' 2 ' 6 ' 1]`. In tổng và trung bình cộng (số thực, hai chữ số lẻ).

6.2 Đếm trong mảng có bao nhiêu phần tử là số chẵn.

6.3 Đảo ngược mảng 5 phần tử (không dùng mảng phụ).

6.4 In tam giác sao cao 4 dòng bằng cách xây chuỗi `dong` rồi ghép thêm một dấu `*` sau mỗi lần lặp.

6.5 Nhập một chuỗi và in `rong` nếu độ dài bằng 0, ngược lại in độ dài.

Chương 7

Hàm và thủ tục: `fn` và `void`

7.1 Vì sao chia nhỏ chương trình?

Khi chương trình dài ra, ta tách những việc lặp lại thành *hàm* có tên rồi gọi chúng khi cần. Bản gốc nói đây là phần "tự do và tự chế ra hàm gọi": người viết tự đặt tên và quyết định việc cho hàm. Sd chia hai họ, theo việc có trả về giá trị hay không.

Họ	Khung	Thẻ	Khép
Hàm (có kết quả)	`iid: sta(fn) => rtn:`	`rtn` (return)	`fn ed`
Thủ tục (không kết quả)	`iid: sta(void) => unkw:`	`unkw` (unknown)	`void ed`

7.2 Hàm có kết quả: `fn`

Ví dụ 7.1. Hàm cộng hai số

```
imp*ioprt.ed

iid: sta(fn) => rtn:
    sta cong(int a ' int b) -> int {
        iid: ref* a + b ed
    }
fn ed

sta(main){
    iid: sta(int s) => edt: sta(s = cong(3 ' 4)) ed
    iid: sta(pri) => "3 + 4 = /d{s}" ed
    iid: ref* 0 ed
}
```

Quy tắc hàm

1. Tên hàm, danh sách tham số `(kiểu tên ' kiểu tên)`, rồi `-> kiểu-trả-về`.
2. Tham số ngăn cách bằng `'`.
3. Thân hàm kết thúc bằng một lệnh `iid: ref* GIÁ-TRỊ ed` ở cấp ngoài cùng (lỗi `E207` nếu thiếu).
4. Các hàm đặt ở ngoài `sta(main)`; thứ tự các khối trên cùng không quan trọng[ĐX], hàm đặt sau `main` vẫn gọi được.
5. Tham số kiểu số, chuỗi, `bool` được truyền *theo giá trị* (hàm chỉ nhận bản sao). Mảng được truyền *theo địa chỉ*[ĐX]: sửa phần tử trong hàm làm đổi mảng của người gọi.

7.2.1 Đệ quy

Hàm được phép gọi chính nó. Mọi hàm đệ quy cần một *điểm dừng* để chuỗi gọi kết thúc.

Ví dụ 7.2. Giai thừa bằng đệ quy

```
imp*ioprt.ed

iid: sta(fn) => rtn:
    sta giaithua(int n) -> int {
        iid: sta(bolean) => cdn,tf:
            sta(n <= 1) {
                iid: ref* 1 ed
            }
        bln ed
        iid: ref* n * giaithua(n - 1) ed
    }
fn ed

sta(main){
    iid: sta(pri) => "5! = /d{giaithua(5)}" ed
    iid: ref* 0 ed
}
```

Kết quả: `5! = 120`. Nhánh `n <= 1` là điểm dừng.

7.2.2 Truyền mảng

Ví dụ 7.3. Hàm tính tổng mảng

```
imp*ioprt.ed

iid: sta(fn) => rtn:
    sta tong(int a[5]) -> int {
        iid: sta(int s) => edt: sta(s = 0) ed
        iid: sta(for) => cdn,chck:
            dbl sta(int i = 0 ' i < 5 ' i++) {
                iid: sta(s = s + a[i]) ed
            }
        dbl ed
        iid: ref* s ed
    }
fn ed

sta(main){
    iid: sta(int d[5]) => edt: sta(d = [1 ' 2 ' 3 ' 4 ' 5]) ed
    iid: sta(pri) => "Tong = /d{tong(d)}" ed
    iid: ref* 0 ed
}
```

7.3 Thủ tục: `void`

Thủ tục làm một việc nhưng không trả về giá trị. Bản gốc mô tả khối `void` là "kiểu ngoài các dữ liệu, không được đọc trừ khi được gọi". Nghĩa là phần thân của thủ tục *chỉ chạy khi có lệnh* `call`; chỉ khai báo thôi thì không có gì xảy ra.

Ví dụ 7.4. Thủ tục chào và lệnh call

```
imp*ioprt.ed

iid: sta(void) => unkw:
    sta chaomung(str ten) {
        iid: sta(pri) => "Xin chao, /s{ten}!" ed
```

```
    }
void ed

sta(main){
    iid: sta(pri) => "Bat dau" ed
    iid: call chaomung("An") ed
    iid: call chaomung("Binh") ed
    iid: ref* 0 ed
}
```

Kết quả:

```
Bat dau
Xin chao, An!
Xin chao, Binh!
```

`call` và gọi trong biểu thức[ĐX]

Thủ tục được gọi như một lệnh: `iid: call ten(...) ed`. Hàm có kết quả được dùng ngay trong biểu thức, như `cong(3 ' 4)`. Nếu bạn muốn gọi hàm nhưng bỏ kết quả, cũng viết `iid: call ten(...) ed`.

Tóm tắt chương

Hàm: `iid: sta(fn) => rtn: … fn ed`, có `-> kiểu` và `ref*` ở cuối. Thủ tục: `iid: sta(void) => unkw: … void ed`, gọi bằng `call`. Tham số ngăn bằng `'`.

Bài tập

7.1 Viết hàm `lonnhat(int a ' int b) -> int` và dùng nó để tìm số lớn nhất trong ba số.

7.2 Viết hàm `lachinhphuong(int n) -> bool` rồi in các số chính phương từ 1 đến 50.

7.3 Viết thủ tục `invach(int n)` in một dòng gồm `n` dấu `-`.

7.4 Viết hàm đệ quy `fib(int n) -> int` trả về số Fibonacci thứ `n` (`fib(0)=0`, `fib(1)=1`).

Chương 8

Cấu trúc dữ liệu: `struct`

8.1 Gom nhiều thông tin thành một

Một sinh viên có tên, tuổi, điểm. Thay vì ba biến rời, ta gom thành một *cấu trúc*. Khối `struct` thuộc *họ đơn*: chỉ có một thẻ `dta` (data) và khép bằng `strt ed`.

Ví dụ 8.1. Khai báo và dùng cấu trúc

```
imp*ioprt.ed

iid: sta(struct) => dta:
    sta SinhVien { str ten ' int tuoi ' float diem }
strt ed

sta(main){
    iid: sta(SinhVien sv) ed
    iid: sta(sv.ten = "An") ed
    iid: sta(sv.tuoi = 19) ed
    iid: sta(sv.diem = 8.5) ed
    iid: sta(pri) => "/s{sv.ten}, /d{sv.tuoi} tuoi, diem /f{sv.diem}" ed
    iid: ref* 0 ed
}
```

Kết quả: `An, 19 tuoi, diem 8.50`.

Quy tắc cấu trúc

1. Các trường ngăn bằng `'`, mỗi trường là `kiểu tên`; **tên trường không được trùng nhau** (lỗi `E302`).
2. Dùng tên cấu trúc như một kiểu: `sta(SinhVien sv)`.
3. Truy cập trường bằng dấu chấm: `sv.ten`.
4. Gán cấu trúc cho cấu trúc cùng kiểu là sao chép toàn bộ trường[ĐX].

Struct khác mảng

Bản gốc viết `sta arr[int a(10) ' str("hello world") ' float a(1.4)]`, trộn khuôn mảng với khuôn cấu trúc và dùng hai trường cùng tên `a`. Sd 2 tách rõ: `[ ]` chỉ dùng cho mảng (các phần tử cùng kiểu), còn `{ }` bọc các trường khác kiểu của cấu trúc.

8.2 Mảng các cấu trúc

Ví dụ 8.2. Tìm sinh viên điểm cao nhất

```
imp*ioprt.ed

iid: sta(struct) => dta:
    sta SinhVien { str ten ' float diem }
strt ed
```

```

sta(main){
    iid: sta(SinhVien ds[3]) ed
    iid: sta(ds[0].ten = "An") ed
    iid: sta(ds[0].diem = 7.0) ed
    iid: sta(ds[1].ten = "Binh") ed
    iid: sta(ds[1].diem = 9.0) ed
    iid: sta(ds[2].ten = "Chi") ed
    iid: sta(ds[2].diem = 8.0) ed
    iid: sta(int vt) => edt: sta(vt = 0) ed
    iid: sta(for) => cdn,chck:
        dbl sta(int i = 1 ' i < 3 ' i++) {
            iid: sta(bolean) => cdn,tf:
                sta(ds[i].diem > ds[vt].diem) {
                    iid: sta(vt = i) ed
                }
            bln ed
        }
    dbl ed
    iid: sta(pri) => "Cao nhat: /s{ds[vt].ten}" ed
    iid: ref* 0 ed
}
```

Kết quả: `Cao nhat: Binh`.

8.3 Cấu trúc và hàm

Cấu trúc có thể làm tham số hoặc kiểu trả về của hàm (truyền theo giá trị, tức bản sao).

Ví dụ 8.3. Hàm nhận một cấu trúc

```
imp*ioprt.ed

iid: sta(struct) => dta:
    sta Diem { int x ' int y }
strt ed

iid: sta(fn) => rtn:
    sta tong(Diem p) -> int {
        iid: ref* p.x + p.y ed
    }
fn ed

sta(main){
    iid: sta(Diem p) ed
    iid: sta(p.x = 3) ed
    iid: sta(p.y = 4) ed
    iid: sta(pri) => "x + y = /d{tong(p)}" ed
    iid: ref* 0 ed
}
```

Tóm tắt chương

`struct` là khối họ đơn: thẻ `dta`, khép `strt ed`. Trường ngăn bằng `'` và không trùng tên. Truy cập bằng dấu chấm; có thể lập mảng cấu trúc.

Bài tập

8.1 Định nghĩa cấu trúc `PhanSo` gồm `tu` và `mau` (nguyên). Viết hàm cộng hai phân số (chưa cần rút gọn).

8.2 Lập mảng 3 `SinhVien`, nhập tên và điểm từ bàn phím, in điểm trung bình lớp.

8.3 Tìm lỗi trong khai báo `sta Xe { str hang ' int doi ' str hang }`.

8.4 Vì sao `SinhVien.ten = "An"` (dùng tên kiểu thay vì tên biến) là sai?

Chương 9

Điểm lỗi và gỡ lỗi

9.1 Ba loại sự cố

Chương trình có thể sai theo ba cách khác nhau, và cách xử lý cũng khác nhau.

Loại	Khi nào xảy ra	Mã
Lỗi cú pháp, ngữ nghĩa	khi biên dịch, chương trình chưa chạy	`E1xx`, `E2xx`, `E3xx`
Lỗi khi chạy	chương trình đang chạy thì gặp tình huống cấm	`E4xx`
Cảnh báo	mã hợp lệ nhưng đáng ngờ, vẫn chạy được	`Wxxx`

Bảng mã đầy đủ ở Phụ lục C.

9.2 Điểm lỗi

Bản gốc có một câu ghi chú ngay cạnh lệnh `ref*`: khi mã chạy sai logic, nó sẽ "chạy ra giá trị điểm lỗi". Sd 2 biến ý đó thành quy ước:

Quy ước điểm lỗi[ĐX]

- `ref* 0` là thành công.
- `ref* N` với `N` từ 1 đến 254 là mã lỗi *do chương trình tự chọn*.
- Khi gặp lỗi khi chạy, hệ thống in `[LOI Exxx @dong N]` kèm mô tả, rồi kết thúc với mã `255`.

Cách phổ biến nhất để tự bảo vệ chương trình là kiểm tra dữ liệu ngay sau khi nhập và thoát bằng mã lỗi của bạn:

Ví dụ 9.1. Kiểm tra dữ liệu nhập và trả điểm lỗi

```
imp*ioprt.ed

sta(main){
    iid: sta(int tuoi) ed
    iid: sta(pri) => "Nhap tuoi:" ed
    iid: sta(ipt) => edt: sta(tuoi) ed
    iid: sta(bolean) => cdn,tf:
        sta(tuoi < 0 or tuoi > 150) {
            iid: sta(pri) => "Tuoi khong hop le" ed
            iid: ref* 2 ed
        }
    bln ed
    iid: sta(pri) => "Tuoi hop le: /d{tuoi}" ed
    iid: ref* 0 ed
}
```

9.3 Đọc thông báo lỗi

Một thông báo có bốn phần: loại và mã lỗi, dòng xảy ra, mô tả, gợi ý.

Ví dụ thông báo

```
[LOI E301 @dong 3] bien 'b' chua khai bao
    iid: sta(pri) => "a = /d{b}" ed
    goi y: ban co muon dung 'a' khong?
```

9.4 Thực hành: tìm lỗi trong chương trình sau

Ví dụ 9.2. Chương trình có ba vấn đề (dùng để luyện tìm lỗi)

```
imp*ioprt.ed

sta(main){
    iid: sta(int a) => edt: sta(a = 3) ed
    iid: sta(pri) => "a = /d{b}" ed
    iid: sta(while) => cdn,edt:
        dbl sta(a > 0) {
            iid: sta(pri) => "lap" ed
        }
    dbl ed
}
```

Ba vấn đề: (1) `E301`, dùng biến `b` chưa khai báo; (2) `W201`, vòng `while` không chỉnh `a` nên chạy mãi; (3) `E204`, khung `main` thiếu `iid: ref* ... ed` ở cuối.

9.5 Quy trình gỡ lỗi năm bước

1. **Đọc mã lỗi** và tra Phụ lục C; đừng đoán.
2. **Đến đúng dòng** được báo, nhưng nhìn cả dòng trước đó (lỗi thiếu chốt thường hiện ở dòng sau).
3. **Thu hẹp**: chèn lệnh `pri` in giá trị các biến để biết chương trình đi tới đâu.
4. **Sửa một chỗ** rồi chạy lại; sửa nhiều chỗ cùng lúc thì không biết chỗ nào có tác dụng.
5. **Ghi lại** nguyên nhân để lần sau không lặp lại.

Danh sách kiểm tra trước khi chạy

- Mỗi `iid:` một dòng đều có `ed` ở cuối.
- Mỗi khối mở đều có chốt khép đúng loại: `dbl`, `strt`, `void`, `bln`, `fn`.
- Số `{` bằng số `}`.
- Mỗi `\\` mở ghi chú đều có `\\` đóng.
- `main` kết thúc bằng `ref*`; hàm có kết quả kết thúc bằng `ref*`.

Tóm tắt chương

Lỗi chia ba loại: biên dịch, chạy, cảnh báo. `ref* 0` là thành công, `ref* N` là điểm lỗi do bạn chọn, lỗi khi chạy trả `255`. Gỡ lỗi theo năm bước: đọc mã, đến dòng, thu hẹp, sửa một chỗ, ghi lại.

Bài tập

9.1 Chương trình thiếu chốt `ed` ở một lệnh `pri`. Bạn dự đoán mã lỗi nào? Tra Phụ lục C để kiểm tra.

9.2 Viết chương trình nhập hai số và in thương; nếu số chia bằng 0 thì in `chia cho 0` và kết thúc bằng `ref* 3`.

9.3 Mảng `a[5]`, lệnh `sta(pri) => "/d{a[5]}"` gây lỗi gì khi chạy? Vì sao?

9.4 Viết lại vòng `while` không chạy mãi: `n = 10; while n != 0 { pri n }` bằng cú pháp Sd 2 và sửa cho dừng được.

Phần II

Sd++ nâng cao

Chương 10

Sd++ và ranh giới `ssta` /

10.1 Sd++ là gì?

Sd++ là phiên bản mở rộng của Sd cho lập trình hệ thống. Mọi thứ bạn đã học ở Phần I (khung `sta`, lệnh `iid`, các khối, hàm, cấu trúc) vẫn dùng nguyên vẹn trong Sd++. Sd++ bổ sung bốn thứ:

1. **Ranh giới tuyệt đối**: `ssta` ở đầu, `##` ở cuối.
2. **Bộ cờ chốt**: `$`, `₫`, `€`, `$$`, `#` cho từng loại phạm vi.
3. **Bộ nhớ cấp thấp**: con trỏ `@`, `*`, cấp phát Heap.
4. **Siêu lập trình**: `define` đổi tên, `defk` tự tạo cờ chốt, và khối chọn lọc tự kiểm tra lại `ids sb` với Reback.

Quan hệ Sd và Sd++[ĐX]

Một chương trình Sd hợp lệ trở thành chương trình Sd++ hợp lệ khi bọc nó trong `ssta { ... }` và kết thúc bằng `##`. Trong Sd++, `ed` được hiểu như cờ `$` ở cấp câu lệnh. Nghĩa là *không có gì phải học lại*; Phần II chỉ thêm công cụ.

10.2 Chiếc hộp bảo mật

Mọi chương trình Sd++ như một chiếc hộp. `ssta` (Super Statement) là nắp mở, đặt ở đầu tệp, thiết lập ngữ nghĩa và bộ nhớ cho cả chương trình. `##` là nắp đóng, đặt ở cuối tệp: khi bộ biên dịch hay bộ chạy chạm `##`, toàn bộ Call Stack bị hủy, thanh ghi được dọn, bộ nhớ Heap và Stack được giải phóng và máy trở về trạng thái ban đầu. Nhờ nắp đóng này, dù chương trình có lỗi nặng thì tài nguyên cũng không bị bỏ sót.

Ví dụ 10.1. Khung tối thiểu của chương trình Sd++

```
ssta {
    */ toan bo ma nguon nam o day
}
##
```

Sau ## không còn gì

`##` phải là mã cuối cùng của tệp; phía sau chỉ được có khoảng trắng và ghi chú. Thiếu `##` là lỗi `E208`. `##` cho phép ngắt toàn bộ chương trình mà không cần đếm xem còn bao nhiêu ngoặc chưa đóng.

10.3 Bộ cờ chốt

Trong Sd++, "chốt đúng cờ cho đúng phạm vi" là nguyên tắc chính. Bạn không rải dấu chốt khắp nơi; chỉ chốt ở nơi một câu lệnh hoặc một phạm vi thực sự kết thúc.

Cờ	Tên	Dùng để
`$`	Chốt câu lệnh (scope)	chốt một câu lệnh thực thi trong khung
`đ`	Chốt toàn cục trần	chốt khai báo biến toàn cục nằm ngoài mọi khung
`€`	Chốt khung ngoài	chốt cả khối `define[...]{ ... }`; áp dụng cơ chế "kết trước, nạp định nghĩa sau"
`$$`	Nhảy có điều kiện	trong `ids sb`: Reback, quay lại xét điều kiện; ngoài khung: dừng khẩn cấp
#	Trạm phân đoạn	niêm phong khối `defk`; điểm neo để phục hồi sau lỗi
##	Kết thúc tất cả	đóng toàn bộ chương trình, dọn sạch tài nguyên

Tính ngắn gọn của một lần chốt

Mỗi cờ chốt chỉ *một lần* cho một câu lệnh hay phạm vi. Một dòng khai báo toàn cục chốt bằng `đ` đúng một lần ở cuối dòng; một khối `define` chốt bằng `€` đúng một lần ở cuối khối.

`$$` ngoài khối sb là dừng khẩn cấp

Khi `$$` xuất hiện ngoài một khối `ids sb` nó trở thành *Hard Halt*: dừng cả hệ thống ngay lập tức (kiểu "Kernel Spinlock"). Đừng dùng `$$` ngoài `ids sb` trừ khi bạn thực sự muốn dừng khẩn cấp.

10.4 Ghi chú trong Sd++

Sd++ dùng được cả hai kiểu ghi chú: `*/` cho ghi chú một dòng (từ `*/` đến hết dòng) và cặp `\\ ... \\` cho ghi chú nhiều dòng. Nên chọn một kiểu cho mỗi tệp để thống nhất.

10.5 Chương trình Sd++ đầu tiên

Ví dụ 10.2. Hello Sd++

```
ssta {
    imp*ioprt.$

    define[
        ids.edt: text = "main",
        pri.edt: text = "print"
    ]{
        main: {
            ids.pri: "Chao mung ban den voi Sd++!" $
            ref*0 $:
        } $
    } €
}
##
```

Cách đọc:

- `ssta { ... }` và `##` là chiếc hộp.
- `imp*ioprt.$` nạp thư viện (chốt `$` thay cho `.ed` của Sd).
- `define[ ... ]{ ... } €` là khung ngoài, sẽ học ở chương 11.
- `ids.pri: "..." $` là lệnh in *dạng phẳng*, tương đương `iid: sta(pri) => "..." ed` của Sd.

- `ref*0 $:` kết thúc khung `main`.

Tóm tắt chương

Sd++ = Sd + ranh giới `ssta/##` + bộ cờ chốt + bộ nhớ cấp thấp + siêu lập trình. `ed` là `$` ở cấp câu lệnh. `$$` ngoài `sb` là dừng khẩn cấp.

Bài tập

10.1 Viết chương trình Sd++ in ra hai dòng bằng `ids.pri`.

10.2 Điền cờ chốt thích hợp (`$`, `₫`, `€`): (a) `ids.g: x = 5 ?` (b) `ids.pri: "a" ?` (c) `define[ ... ]{ ... } ?`

10.3 Điều gì xảy ra khi tệp Sd++ thiếu `##`?

Chương 11

Biến toàn cục trần và khung `define`

11.1 Biến toàn cục trần

Biến khai báo *ngoài* mọi khung, gọi là biến toàn cục trần, dùng được ở mọi nơi trong chương trình. Khai báo dùng `ids.g:` và chốt bằng `đ`; gọi lại bằng `ids(tên).g`.

Ví dụ 11.1. Biến toàn cục và biến cục bộ

```
ssta {
    imp*ioprt.$

    ids.g: ten_may = "May chu A" đ
    ids.g: nhiet_do = -20 đ

    define[
        ids.edt: text = "main",
        pri.edt: text = "print"
    ]{
        main: {
            iptedt.int t = ids(nhiet_do).g $
            ids.pri: "May: /s{ids(ten_may).g}" $
            ids.pri: "Nhiet do: /d{t}" $
            ref*0 $:
        } $
    } €
}
##
```

Dòng `iptedt.int t = ids(nhiet_do).g $` khai báo biến cục bộ `t` (chốt bằng `$`) và gán giá trị từ biến toàn cục. Kết quả theo đặc tả:

```
May: May chu A
Nhiet do: -20
```

Quy tắc biến toàn cục[ĐX]

1. Tên biến cục bộ không được trùng tên biến toàn cục (lỗi `E302`), để cách đọc `ids(x).g` và `x` không bao giờ nhập nhằng.
2. Biến toàn cục có thể là số, chuỗi, mảng (`sta.arr.g`) hay con trỏ (`ptr.g`); cả bốn đều chốt bằng `đ`.
3. Mảng toàn cục ngăn các phần tử bằng `'`, như mọi nơi khác: `sta.arr.g: so = [10 ' 20 ' 30] đ`. Dạng ngăn bằng dấu phẩy của đặc tả cũ vẫn được đọc, kèm cảnh báo `W202`.

11.2 Khung `define[...]{ ... } €`

Khung ngoài làm hai việc: *đổi tên* cho các thành phần chuẩn (chương trình chính, lệnh in) và *chứa* chương trình chính.

Giải phẫu khung define

```
define[
    ids.edt: text = "main",      <- dat ten "main" cho khung chuong trinh chinh
    pri.edt: text = "print"      <- dat ten "print" cho lenh in
]{
    main: {                      <- khung chinh, ten theo dong tren
        ...
    } $
} €                              <- chot khung ngoai
```

Danh sách trong `[ ]` gồm các *định nghĩa*, ngăn cách bằng dấu phẩy. Mỗi định nghĩa có dạng `loại.edt: text = "tên"`. Trong cặp `{ }` tiếp theo là nội dung khung. Cách đọc từ các ví dụ gốc: sau khi `pri.edt: text = "in_ra"`, bạn có thể viết `in_ra: "..." $` thay cho `ids.pri: "..." $`; dạng đầy đủ `ids.pri:` luôn dùng được.

Ví dụ 11.2. Đổi tên lệnh in thành in_ra

```
ssta {
    imp*ioprt.$

    define[
        ids.edt: text = "main",
        pri.edt: text = "in_ra"
    ]{
        main: {
            in_ra: "Dung ten moi" $
            ids.pri: "Dung ten day du" $
            ref*0 $:
        } $
    } €
}
##
```

11.2.1 Cơ chế "kết trước, nạp định nghĩa sau"

Cờ `€` nằm ở *cuối* khung, nhưng các định nghĩa trong `define[ ]` có hiệu lực cho cả thân khung. Vì vậy trình biên dịch làm hai lượt: lượt 1 đọc đến `€` để biết khung kết thúc ở đâu; lượt 2 nạp các định nghĩa rồi mới phân tích thân khung. Nhờ đó bạn dùng được tên mới ngay từ dòng đầu của thân khung.

11.3 Đối chiếu dạng phẳng của Sd++ với Sd

Sd (chương 1–9)	**Sd++ dạng phẳng**
`iid: sta(pri) => "x" ed`	`ids.pri: "x" $`
`iid: sta(int a) => edt: sta(a = 5) ed`	`iptedt.int a = 5 $`
`iid: sta(ptr p) => edt: sta(p = @a) ed`	`iptedt.ptr p = @a $`
khai báo ngoài mọi khung	`ids.g: a = 5 đ`
`iid: ref* 0 ed`	`ref*0 $:`
`ed` (kết thúc câu lệnh)	`$`
`sta(main){ ... }`	`define[...]{ main: { ... } $ } €`

Hai cách viết đứng cạnh nhau được trong cùng một tệp Sd++; chọn một cách cho cả dự án để dễ đọc.

Tóm tắt chương

Biến toàn cục trần: khai báo `ids.g: tên = giá-trị đ`, gọi bằng `ids(tên).g`. Khung ngoài `define[...]{...} €` đổi tên thành phần chuẩn và chứa `main`; trình biên dịch nạp định nghĩa theo hai lượt.

Bài tập

11.1 Khai báo hai biến toàn cục `ten` và `tuoi`, in ra bằng định dạng `/s{ }` và `/d{ }`.

11.2 Vì sao biến toàn cục chốt bằng `đ` còn biến trong khung chốt bằng `$`?

11.3 Viết lại bằng Sd: `iptedt.int a = 5 $` và `ids.pri: "hi" $`.

Chương 12

Con trỏ và bộ nhớ Heap

12.1 Con trỏ là gì?

Mỗi biến nằm ở một vị trí trong bộ nhớ ảo, gọi là *địa chỉ*. Con trỏ là biến chứa một địa chỉ. Sd++ dùng hai ký hiệu:

- `@x` (lấy địa chỉ): cho địa chỉ của biến hoặc mảng `x`.
- `*p` (giải tham chiếu): đọc hoặc ghi giá trị tại địa chỉ mà `p` đang trỏ tới.

Hình dung bộ nhớ

```
 bien temp           con tro p
+--------+          +--------+
|   20   | <------- |  @temp |        *p  la gia tri 20
+--------+          +--------+        p   la dia chi cua temp
  dia chi A           dia chi B       @p  la dia chi B
```

Ví dụ 12.1. Đọc và ghi qua con trỏ

```
ssta {
    imp*ioprt.$

    define[
        ids.edt: text = "main",
        pri.edt: text = "print"
    ]{
        main: {
            iptedt.int temp = 20 $
            iptedt.ptr p = @temp $
            *p = *p + 5 $
            ids.pri: "temp = /d{temp}" $
            ref*0 $:
        } $
    } €
}
##
```

Kết quả: `temp = 25`. Lệnh `*p = *p + 5` sửa chính biến `temp` vì `p` trỏ vào nó.

Con trỏ toàn cục

Con trỏ cũng khai báo trần ngoài khung: `ptr.g: p_nhiet = @nhiet_do đ`. Hãy khai báo biến mà con trỏ trỏ tới *trước* con trỏ.

12.2 Bộ nhớ Heap

Biến thường tự mất khi khung đóng. Khi cần vùng nhớ sống lâu hơn, hoặc kích thước chỉ biết lúc chạy, ta xin bộ nhớ từ *Heap*:

- `sta.alloc(n)` cấp phát `n` byte và trả về con trỏ tới vùng đó.
- `sta.free(p)` trả vùng nhớ lại cho hệ thống.

Một *ô nhớ* có 8 byte[ĐX], nên `sta.alloc(32)` cho bốn ô. Ô thứ `k` tính từ con trỏ `h` là `*(h + k)`; `*h` là ô đầu tiên (`k = 0`).

Ví dụ 12.2. Cấp phát, dùng và giải phóng

```
ssta {
    imp*ioprt.$
    imp*mem.$

    define[
        ids.edt: text = "main",
        pri.edt: text = "print"
    ]{
        main: {
            iptedt.ptr h = sta.alloc(32) $
            iid: sta(for) => cdn,chck:
                dbl sta(int i = 0 ' i < 4 ' i++) {
                    iid: sta(*(h + i) = i * i) ed
                }
            dbl ed
            ids.pri: "o thu 3 = /d{*(h + 3)}" $
            sta.free(h) $
            ref*0 $:
        } $
    } €
}
##
```

Kết quả: `o thu 3 = 9`. Ví dụ này đặt lệnh Sd (`iid: ... ed`) cạnh lệnh dạng phẳng, như đã nói ở chương 11.

12.3 Quy tắc an toàn khi dùng bộ nhớ

Bốn quy tắc

1. **Cấp phát thì phải giải phóng.** Mỗi `sta.alloc` cần một `sta.free` tương ứng; bỏ sót là rò rỉ bộ nhớ (cảnh báo `W302`).
2. **Sau khi `sta.free(p)`, `p` trở thành con trỏ rỗng**[ĐX]. Giải tham chiếu con trỏ rỗng là lỗi `E405`.
3. **Giải phóng con trỏ rỗng hay giải phóng hai lần** là lỗi `E406`.
4. **Không đọc ghi ngoài vùng đã cấp.** Ví dụ `*(h + 4)` khi chỉ xin 4 ô là lỗi `E408`. Xin quá nhiều bộ nhớ gây `E407`.

Lỗi thường gặp

- Quên `sta.free` rồi lặp lại cấp phát trong vòng lặp: bộ nhớ cạn dần.
- Dùng con trỏ sau khi giải phóng.
- Lẫn `p` (địa chỉ) với `*p` (giá trị).

Tóm tắt chương

`@x` lấy địa chỉ, `*p` đọc/ghi giá trị tại địa chỉ. Heap: `sta.alloc(n)` cấp phát, `sta.free(p)` giải phóng; mỗi `alloc` cần một `free`. Con trỏ rỗng và vượt vùng là lỗi khi chạy.

Bài tập

12.1 Viết chương trình hoán đổi giá trị hai biến `x`, `y` thông qua hai con trỏ.

12.2 Cấp phát 4 ô, điền `10, 20, 30, 40`, in tổng, rồi giải phóng.

12.3 Tìm lỗi: `sta.free(h) $` rồi `*h = 1 $`.

Chương 13

Khối chọn lọc `ids sb` và Reback

13.1 Ý tưởng: tự kiểm tra lại

Ở Phần I, muốn lặp lại việc xét điều kiện ta dùng `while`. Sd++ có thêm một khối khác: *khối chọn lọc* `ids sb(biến)` (selection block). Nó giống `bolean` ở chỗ có nhiều nhánh, nhưng có thêm một khả năng: sau khi một nhánh *sửa* biến, nó có thể **quay lại đầu khối và xét lại điều kiện** với giá trị mới. Hành vi đó gọi là *Reback*, đánh dấu bằng cờ `$$` ở cuối nhánh.

13.2 Cú pháp

Khuôn khối chọn lọc

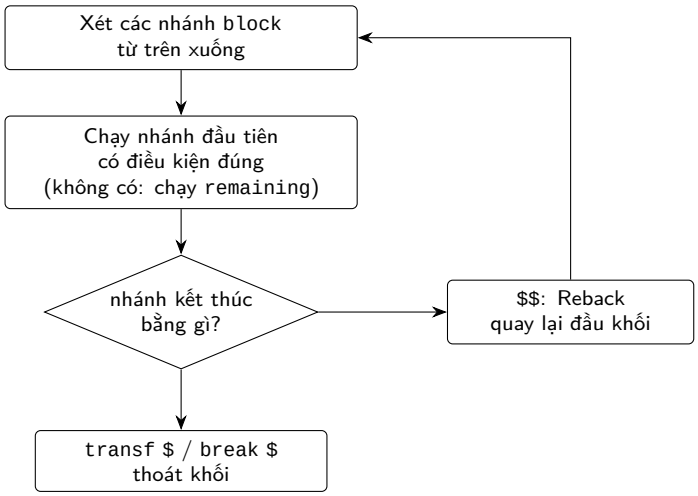
```
ids sb(BIEN) {
    block DIEU-KIEN-1:
        ...
        transf $            <- xong, thoat khoi
    block DIEU-KIEN-2:
        ...sua BIEN...
        $$                  <- Reback: quay lai xet lai tu dau
    remaining:              <- khi khong dieu kien nao dung
        ...
        break $             <- thoat khoi ngay
} $
```

Thành phần	Ý nghĩa
`ids sb(biến)`	mở khối; `biến` là biến mà các điều kiện dựa vào
`block cdk:`	một nhánh, chạy nếu điều kiện `cdk` đúng
`remaining:`	nhánh còn lại, chạy khi không nhánh `block` nào đúng
`transf $`	kết thúc nhánh và *thoát khối* (hoàn tất)
`break $`[ĐX]	thoát khối ngay, dùng ở nhánh không xử lý được gì
`$$`	kết thúc nhánh và *Reback*: quay lại đầu khối

Điểm làm gọn so với bản gốc[ĐX]

Bản gốc dùng `$$` cho cả hai việc: Reback ở nhánh sửa lỗi và "kết thúc" ở `break $$`; lại viết `transf $ $` với hai dấu tách rời. Điều đó làm một cờ có hai nghĩa. Sd++ 2 tách: `$$` *chỉ* là Reback; thoát khối dùng `transf $` hoặc `break $`. Dạng cũ vẫn đọc được, kèm cảnh báo `W202`.

13.3 Sơ đồ hoạt động



13.4 Ví dụ: hệ thống tự sưởi

Ví dụ 13.1. Nhiệt độ âm tự cộng cho đến khi ổn định

```
ssta {
    imp*ioprt.$

    define[
        ids.edt: text = "main",
        pri.edt: text = "print"
    ]{
        main: {
            iptedt.int temp = -25 $
            ids.pri: "Nhiet do dau: /d{temp}" $
            ids sb(temp) {
                block temp > 0:
                    ids.pri: "On dinh: /d{temp}" $
                    transf $
                block temp < 0:
                    ids.pri: "Am (/d{temp}), dang suoi" $
                    temp = temp + 20 $
                    $$
                remaining:
                    ids.pri: "Bang 0, dua ve 1" $
                    temp = 1 $
                    $$
            } $
            ref*0 $:
        } $
    } €
}
##
```

Theo dõi từng bước với `temp = -25`:

Lượt	Nhánh được chọn	Kết quả
1	`temp < 0`	in "Am (-25)...", `temp` thành `-5`, Reback
2	`temp < 0`	in "Am (-5)...", `temp` thành `15`, Reback
3	`temp > 0`	in "On dinh: 15", `transf` thoát khối

13.5 Quy tắc chống vòng vô hạn

Giống `while`, Reback có nguy cơ lặp mãi. Sd++ 2 thêm hai lớp bảo vệ[ĐX]:

An toàn của Reback

1. **Kiểm tra lúc biên dịch.** Nhánh kết thúc bằng `$$` phải sửa biến của `sb` (trực tiếp, hoặc qua con trỏ tới biến đó). Nếu trình biên dịch không thấy dấu hiệu nào: cảnh báo `W203`.
2. **Giới hạn lúc chạy.** Mỗi lần khối `sb` được kích hoạt chỉ cho phép tối đa 65 536 lần Reback; vượt quá là lỗi `E402`.

13.6 Đối chiếu với `while`

Khối `sb` ở trên tương đương với vòng `while` chứa rẽ nhánh. Chọn `sb` khi mỗi giá trị của biến cần một cách xử lý riêng và xử lý đó có thể tự đưa biến về trạng thái tốt; chọn `while` khi chỉ cần lặp đơn giản.

Tóm tắt chương

`ids sb(biến)` có các nhánh `block cdk:` và `remaining:`. Kết thúc nhánh bằng `transf $` (hoàn tất), `break $` (thoát) hoặc `$$` (Reback, xét lại). Có hàng rào chống lặp vô hạn: `W203` và `E402`.

Bài tập

13.1 Viết khối `ids sb` cho điểm `score = -15`: nếu điểm không dương thì cộng 30 rồi Reback; khi dương in “Duyet diem thanh cong”.

13.2 Viết lại bài 13.1 bằng vòng `while` của Sd.

13.3 Vì sao nhánh kết thúc bằng `$$` mà không sửa biến của `sb` là nguy hiểm?

Chương 14

Siêu lập trình: defk và cờ chốt tùy biến

14.1 Tự tạo cờ chốt

Cờ chốt `$`, `€`, `$$` đủ dùng, nhưng đôi khi bạn muốn một ký hiệu riêng cho dự án. Khối `defk` (define key) cho phép tạo một cờ chốt mới, ví dụ `¥`, rồi niêm phong quy tắc bằng cờ `#` để không ai ghi đè được.

Ví dụ 14.1. Tạo và dùng cờ chốt `¥`

```
ssta {
    imp*ioprt.$

    defk<ids.end: text "¥"> {
        init ids.edt ("¥" endlogic == "$"+"€"+"$$")
        func * end.sta * end.iid #
    }

    define[
        ids.edt: text = "main",
        pri.edt: text = "print"
    ]{
        main: {
            init ¥
            ids.pri: "Dung chot tuy bien" ¥
            ref*0 ¥:
        } ¥
    } €
}
##
```

14.2 Đọc từng phần của defk

Phần này là *cách đọc của người biên soạn* dựa trên mô tả gốc; tác giả có thể chỉnh lại.

Thành phần	Ý nghĩa
`defk<ids.end: text "¥"> {`	mở khối, khai báo ký hiệu kết thúc mới là chữ `¥`
`init ids.edt (...)`	điều kiện kích hoạt: `¥` mang logic kết thúc bằng hợp của `$`, `€`, `$$`, nghĩa là ở nơi nào cần một trong ba cờ ấy, `¥` đứng thay được
`func * end.sta * end.iid`	áp dụng cho mọi phần kết thúc của khung `sta` và của lệnh `iid` (dấu `*` nghĩa là "mọi")
`#`	niêm phong: quy tắc vừa khai báo không bị `define` hay `init` nào ghi đè
`init ¥`	bật `¥` trong khung hiện tại

14.3 Quy tắc của defk

Bốn quy tắc

1. Khối `defk` phải kết thúc bằng #; thiếu thì lỗi `E210`.
2. Không được định nghĩa lại các cờ dành riêng `$`, `₫`, `€`, `$$`, #, ##, và không ghi đè một cờ đã niêm phong; vi phạm là lỗi `E209`[ĐX].
3. `defk` phải đứng trước chỗ dùng đầu tiên của cờ mới; trong khung phải có `init ¥` trước khi dùng.
4. Cờ tùy biến nên là một ký tự không có trong mã thường dùng, để trình đọc không nhầm với chuỗi hay toán tử.

Vai trò của #: điểm neo

Ngoài việc niêm phong `defk`, # là *điểm neo phục hồi*: khi trình phân tích gặp lỗi cú pháp, nó bỏ qua phần hỏng cho đến # gần nhất rồi tiếp tục ở trạng thái sạch (Panic Mode Recovery, chương 16). Vì vậy # giúp một lỗi không kéo theo hàng chục thông báo lỗi giả.

14.4 Vùng tự chốt :&

Viết `$` sau từng dòng của Sd++ rất đều tay. Ký hiệu `:&` cho phép bỏ việc đó: bạn khai báo ký hiệu bằng `init.endcomand: ":&" $`, rồi đặt một dòng `:&` ở cuối nhóm lệnh. Trình biên dịch tự thêm một kiểu kết *ẩn* vào mọi lệnh nằm trên dòng `:&`, tính từ ranh giới `:&` trước đó. Dòng `:&` chỉ trình biên dịch đọc, nó không sinh lệnh nào khi chạy.

Ví dụ 14.2. Một vùng tự chốt đến cuối tệp

```
ssta {
    init.endcomand: ":&" $
    ids.g: lan = 3
    ids.pri: "Xin chao"
    ids.pri: "Gia tri: /d{lan}"
    :&
}
##
```

Kết quả: `Xin chao` rồi `Gia tri: 3`. Ranh giới `:&` được phép đứng ngay trước ##.

Phần *dưới* dòng `:&` không được thêm gì: nó chạy theo quy tắc mặc định của Sd++, tức là bạn phải tự viết cờ chốt.

Ví dụ 14.3. Hai vùng: tự chốt rồi chốt tay

```
ssta {
    init.endcomand: ":&" $
    ids.pri: "Vung 1: tu chot"
    :&
    ids.pri: "Vung 2: chot tay" $
    ids.pri: "Van vung 2" $
}
##
```

Ví dụ 14.4. Nhiều ranh giới liên tiếp

```
ssta {
    init.endcomand: ":&" $
    ids.g: a = 4
    :&
    ids.pri: "a = /d{a}" $
    ids.g: b = 5
    :&
    ids.pri: "b = /d{b}" $
    ref*0 $:
}
##
```

Kết quả: `a = 4` rồi `b = 5`. Mỗi dòng `:&` chỉ ảnh hưởng đến các lệnh nằm giữa nó và ranh giới trước.

Quy tắc của `:&`

1. `init.endcomand: ":&" $` khai báo ký hiệu; mặc định cũng là `:&`.
2. Mọi lệnh phía trên một dòng `:&`, lùi đến ranh giới trước, được trình biên dịch thêm kiểu kết ẩn.
3. Vùng sau `:&` cuối cùng viết cờ chốt như thường.
4. Ranh giới được đặt ở cuối tệp, sau `}` của `ssta` và trước ##.

Phần mở rộng của người biên soạn

```
\\ Lệnh viết thẳng trong ssta thành chương trình main ngầm \\
ssta {
    init.endcomand: ":&" $
    ids.pri: "Khong can khung main"
    :&
}
##
```

Các điểm sau là **đề xuất của người biên soạn**[ĐX], không thuộc đặc tả gốc: (a) lệnh viết thẳng trong `ssta` tạo thành chương trình `main` ngầm; (b) khung `main: {` trần; (c) dạng `: $`; (d) lỗi `E213` khi đặt `:&` bên trong một khối; (e) lỗi `E214` khi vừa có `main` vừa có lệnh viết thẳng.

Điều chưa chốt

Hai việc tác giả ngôn ngữ chưa quy định hẳn, nên sách chưa dạy như quy tắc: (1) có nhiều tầng chặt chẽ (Tự động, Cục bộ chặt `$`, Tuyệt đối chặt) hay chỉ "vùng tự chốt rồi vùng viết tay"; (2) việc "suy luận hoãn" kiểu kết cho những lệnh mà trình biên dịch chưa đoán được. Bộ thông dịch hiện chạy theo mô hình hai vùng.

Tóm tắt chương

`defk<...> { init ... func ... # }` tạo cờ chốt tùy biến; `#` niêm phong và làm điểm neo phục hồi. Cờ dành riêng không được định nghĩa lại.

Bài tập

14.1 Viết khối `defk` tạo cờ chốt `¤` theo mẫu của sách.

14.2 Giải thích vì sao `defk` định nghĩa lại `$` là lỗi.

14.3 Viết chương trình Sd++ in ba dòng, dùng một dòng `:&` thay cho ba cờ `$`.

14.4 Viết chương trình có hai vùng: vùng tự chốt in một dòng, vùng chốt tay in một dòng.

Chương 15

Thư viện chuẩn

Thư viện được nạp bằng `imp*tên.ed` (Sd) hoặc `imp*tên.$` (Sd++), đặt ở đầu tệp. Dùng hàm của thư viện chưa nạp là lỗi `E205`.

Thư viện	Chức năng	Hàm chính (chữ ký là đề xuất [ĐX])
`ioprt`	nhập xuất màn hình	`pri`, `ipt` (từ khóa của ngôn ngữ)
`math`	toán học	`math.abs(x)`, `math.pow(x ' y)`, `math.sqrt(x)`, `math.max(a ' b)`, `math.min(a ' b)`
`mem`	bộ nhớ cấp thấp	`mem.memset(p ' giá-trị ' số-ô)`, `sta.alloc(byte)`, `sta.free(p)`
`str`	chuỗi	`str.len(s)`, `str.upper(s)`, `str.lower(s)`, `str.concat(a ' b)`
`sys`	hệ thống	`sys.time()` (giây), `sys.sleep(ms)`
`file`	tệp trên ổ đĩa	`file.write(đường-dẫn ' chuỗi)`, `file.read(đường-dẫn)`
`draw`	vẽ khung ký tự	`draw.rect(x ' y ' rộng ' cao)`

15.1 Toán học

Ví dụ 15.1. Căn, lũy thừa, lớn nhất

```
imp*ioprt.ed
imp*math.ed

sta(main){
    iid: sta(float c) => edt: sta(c = math.sqrt(2.0)) ed
    iid: sta(int p) => edt: sta(p = math.pow(2 ' 10)) ed
    iid: sta(int m) => edt: sta(m = math.max(3 ' 8)) ed
    iid: sta(pri) => "sqrt2=/f3{c} 2^10=/d{p} max=/d{m}" ed
    iid: ref* 0 ed
}
```

Kết quả theo đặc tả: `sqrt2=1.414 2^10=1024 max=8`.

15.2 Tệp

Ví dụ 15.2. Ghi rồi đọc lại một tệp

```
imp*ioprt.ed
imp*file.ed

sta(main){
    iid: call file.write("nhat_ky.txt" ' "Sd++ hoat dong") ed
    iid: sta(str s) => edt: sta(s = file.read("nhat_ky.txt")) ed
    iid: sta(pri) => "Doc lai: /s{s}" ed
    iid: ref* 0 ed
}
```

15.3 Dự án nhiều tệp[ĐX]

Khi chương trình lớn dần, bạn tách thành nhiều tệp và nạp bằng `imp*tên.ed` (Sd) hoặc `imp*tên.$` (Sd++). Quy ước của bộ thông dịch kèm sách:

- `imp*tên` tìm tệp `tên.sd` cùng thư mục với tệp đang chạy; tên trùng thư viện chuẩn thì dùng thư viện chuẩn.
- Một tệp được nạp chỉ nạp một lần; nạp vòng tròn là lỗi `E211`, không tìm thấy tệp là lỗi `E215`.
- Hàm `main` của tệp được nạp bị bỏ qua; chỉ tệp chính (đánh dấu sao trong IDE) chạy `main`.
- IDE có nút "Mở thư mục" để nạp cả dự án; dòng lệnh dùng `node sd.js tệp-chính.sd`.

tien_ich.sd: tệp được nạp

```
iid: sta(fn) => rtn:
    sta gap_doi(int x) -> int {
        iid: ref* x * 2 ed
    }
fn ed
```

bai.sd: tệp chính

```
imp*ioprt.ed
imp*tien_ich.ed

sta(main){
    iid: sta(pri) => "Gap doi 21 = /d{gap_doi(21)}" ed
    iid: ref* 0 ed
}
```

Lỗi trong tệp được nạp báo kèm tên tệp. Cách tổ chức này là đề xuất của người biên soạn, không thuộc đặc tả gốc.

Thời gian và nghỉ

`sys.time()` trả số giây tính từ một mốc cố định, dùng để đo thời gian chạy bằng cách trừ hai lần gọi. `sys.sleep(ms)` tạm dừng chương trình `ms` mili-giây.

Tóm tắt chương

Nạp thư viện bằng `imp*tên`. Bảy thư viện chuẩn: `ioprt`, `math`, `mem`, `str`, `sys`, `file`, `draw`. Hàm gọi dạng `thư-viện.hàm(đối ' đối)`.

Bài tập

15.1 Nhập hai cạnh góc vuông, in cạnh huyền bằng `math.sqrt`.

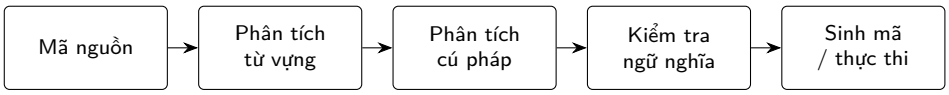
15.2 Ghi một chuỗi vào tệp rồi đọc lại và in ra.

Chương 16

Trình biên dịch Sd và Sd++

Chương này dành cho người muốn hiểu hoặc tự viết trình biên dịch. Đặc tả gốc nhắm tới mã máy x86_64; cách làm thực tế là viết trước một bộ thông dịch tham chiếu, kiểm tra ngữ nghĩa đúng rồi mới sinh mã máy.

16.1 Dây chuyền



16.2 Phân tích từ vựng

Bộ phân tích từ vựng (lexer) cắt mã thành các *token*. Những điểm riêng của Sd:

- `ed`, `dbl`, `strt`, `void`, `bln`, `fn` là từ khóa; `$`, `₫`, `€`, `$$`, `#`, `##` là cờ chốt (khớp dài nhất: `$$` trước `$`, `##` trước `#`).
- `=>` là một token; `==` và `=` là hai token khác.
- Dấu `'` là dấu ngăn mục dữ liệu, dấu `,` ngăn thẻ đặc tả.
- Ghi chú: khi thấy `\\` mở, lexer bỏ qua cho đến `\\` đóng. Nếu chạm cuối tệp mà chưa đóng, báo `E101` kèm dòng mở thay vì nuốt im lặng.

16.3 Phân tích cú pháp

Cú pháp của Sd rất đều đặn: một lệnh là `iid: sta(...) [=> thẻ: ...] ed`; một khối có dòng đầu kết thúc bằng `:` và dòng khép `tên-khép ed`. Vì vậy bộ phân tích dùng một *ngăn xếp các khối đang mở*, kiểm tra quy tắc khép kín như sau:

Giả mã kiểm tra quy tắc khép kín

```
ngan_xep = []
voi moi dong:
    neu dong la dau khoi "iid: sta(K) => the:":
        kiem tra the khop bang (for: cdn,chck ; while: cdn,edt ; ...)
        day khep(K) vao ngan_xep           // dbl / strt / void / bln / fn
    neu dong la "X ed" voi X trong {dbl,strt,void,bln,fn}:
        neu ngan_xep rong        -> loi E202
        neu dinh != X            -> loi E202 (khep sai loai)
        nguoc lai                -> lay dinh ra
het tep:
    neu ngan_xep con phan tu     -> loi E202 (chua khep)
```

16.4 Kiểm tra ngữ nghĩa

Những kiểm tra quan trọng nhất của Sd 2 / Sd++ 2:

1. Mỗi biến được khai báo trước khi dùng (`E301`); không trùng tên (`E302`).
2. Kiểu hợp lệ: không gán chuỗi cho `int`; điều kiện của `bolean` và `while` phải là `bool` (`E303`).
3. `main` và mọi `fn` kết thúc bằng `ref*` (`E204`, `E207`).
4. Vòng `while` phải sửa một biến trong điều kiện (`W201`); nhánh Reback phải sửa biến của `sb` (`W203`).
5. Mỗi `sta.alloc` có `sta.free` cùng khung (`W302`).

16.5 Phục hồi sau lỗi

Khi gặp lỗi cú pháp, một trình biên dịch tốt không dừng ngay mà cố tìm thêm lỗi khác. Sd++ dùng *phục hồi chế độ hoảng loạn* (panic mode): bỏ qua token cho đến một *điểm đồng bộ* rồi phân tích tiếp. Điểm đồng bộ là: dòng khép `tên-khép ed`, cờ `#`, hoặc `##`. Chạm `##` thì dọn sạch trạng thái và kết thúc chắc chắn.

16.6 Lộ trình hiện thực

1. Bộ thông dịch tham chiếu `sdi.py` (Phụ lục G) đã hoàn thành bước 1 đến 4 cho Sd 2; hãy đọc mã của nó trước. Viết lexer và in danh sách token cho các ví dụ của sách.
2. Viết parser cho Phần I, dựng cây cú pháp, chạy bộ kiểm tra khép kín.
3. Viết thông dịch cây cú pháp: biến, phép toán, `pri`, `ipt`, rẽ nhánh, vòng lặp.
4. Thêm hàm, mảng, cấu trúc; chạy lại mọi ví dụ và so với kết quả trong sách.
5. Thêm Sd++: con trỏ, Heap, `sb` với Reback, `define`, `defk`.
6. Cuối cùng mới sinh mã máy hoặc mã trung gian.

Tóm tắt chương

Dây chuyền: từ vựng, cú pháp, ngữ nghĩa, sinh mã. Quy tắc khép kín kiểm tra bằng ngăn xếp. Điểm đồng bộ `tên-khép ed`, `#`, `##` giúp phục hồi sau lỗi. Nên viết thông dịch tham chiếu trước.

Bài tập

16.1 Liệt kê token của dòng `iid: sta(int a) => edt: sta(a = 5) ed`.

16.2 Với mã thiếu hai chữ `bln ed`, trình biên dịch có ngăn xếp nào còn dư vào cuối tệp?

Chương 17

Dự án tổng hợp

17.1 Dự án 1: Sổ điểm lớp (Sd)

Dự án dùng cấu trúc, mảng, hàm, vòng lặp và rẽ nhánh. Chương trình nhập tên và điểm của ba học sinh, in bảng điểm kèm xếp loại và điểm trung bình lớp.

Ví dụ 17.1. Sổ điểm lớp

```
imp*ioprt.ed

iid: sta(struct) => dta:
    sta HocSinh { str ten ' float diem }
strt ed

iid: sta(fn) => rtn:
    sta xeploai(float d) -> str {
        iid: sta(str kq) => edt: sta(kq = "Yeu") ed
        iid: sta(bolean) => cdn,tf:
            sta(d >= 8.0) {
                iid: sta(kq = "Gioi") ed
            }
            sta(d >= 6.5) {
                iid: sta(kq = "Kha") ed
            }
            sta(d >= 5.0) {
                iid: sta(kq = "Trung binh") ed
            }
        bln ed
        iid: ref* kq ed
    }
fn ed

sta(main){
    iid: sta(HocSinh ds[3]) ed
    iid: sta(float tong) ed
    iid: sta(for) => cdn,chck:
        dbl sta(int i = 0 ' i < 3 ' i++) {
            iid: sta(pri) => "Ten hoc sinh /d{i + 1}:" ed
            iid: sta(ipt) => edt: sta(ds[i].ten) ed
            iid: sta(pri) => "Diem:" ed
            iid: sta(ipt) => edt: sta(ds[i].diem) ed
            iid: sta(tong = tong + ds[i].diem) ed
        }
    dbl ed
    iid: sta(pri) => "--- Bang diem ---" ed
    iid: sta(for) => cdn,chck:
        dbl sta(int i = 0 ' i < 3 ' i++) {
            iid: sta(str lx) => edt: sta(lx = xeploai(ds[i].diem)) ed
            iid: sta(pri) => "/s{ds[i].ten} /f{ds[i].diem} /s{lx}" ed
        }
    dbl ed
    iid: sta(pri) => "Trung binh lop: /f{tong / 3.0}" ed
    iid: ref* 0 ed
}
```

Với đầu vào `An 7.0`, `Binh 9.0`, `Chi 4.5`, kết quả theo đặc tả:

```
--- Bang diem ---
An 7.00 Kha
Binh 9.00 Gioi
Chi 4.50 Yeu
Trung binh lop: 6.83
```

17.2 Dự án 2: Giám sát máy chủ tự chữa lành (Sd++)

Dự án dùng biến toàn cục, con trỏ, khối `ids sb` với Reback, cờ tùy biến `¥` và Heap. Khi trạng thái nút máy chủ âm, hệ thống tự cộng cho đến khi đạt 200 (chạy đủ công suất), rồi ghi nhật ký vào Heap.

Ví dụ 17.2. Giám sát và tự chữa lành

```
ssta {
    imp*ioprt.$
    imp*mem.$

    defk<ids.end: text "¥"> {
        init ids.edt ("¥" endlogic == "$"+"€"+"$$")
        func * end.sta * end.iid #
    }

    ids.g: cluster = "Cloud-Cluster-Alpha" đ
    ids.g: node_status = -100 đ

    define[
        ids.edt: text = "main",
        pri.edt: text = "print"
    ]{
        main: {
            init ¥
            iptedt.int status = ids(node_status).g ¥
            iptedt.ptr p = @status ¥
            ids.pri: "== MONITOR: /s{ids(cluster).g} ==" ¥
            ids.pri: "Trang thai ban dau: /d{status}" ¥
            ids sb(status) {
                block status == 200:
                    ids.pri: "[ONLINE] 100% cong suat" ¥
                    transf $
                block status < 0:
                    ids.pri: "[WARNING] su co (/d{status}), reset" ¥
                    *p = *p + 300 ¥
                    $$
                remaining:
                    ids.pri: "[RESET] dua ve 200" ¥
                    *p = 200 ¥
                    $$
            } ¥
            iptedt.ptr log = sta.alloc(16) $
            *log = status $
            ids.pri: "Ghi nhat ky: /d{*log}" $
            sta.free(log) $
            ref*0 ¥:
        } ¥
    } €
}
##
```

Kết quả theo đặc tả:

```
== MONITOR: Cloud-Cluster-Alpha ==
Trang thai ban dau: -100
[WARNING] su co (-100), reset
[ONLINE] 100% cong suat
Ghi nhat ky: 200
```

Nhánh `status < 0` sửa `status` gián tiếp qua con trỏ `p`; trình biên dịch có thể phát `W203` vì không chắc con trỏ trỏ tới biến nào, nhưng ở đây `p` được gán `@status` nên cảnh báo này chỉ là nhắc.

Bài tập

17.1 Mở rộng Dự án 1: thêm in tên học sinh có điểm cao nhất.

17.2 Mở rộng Dự án 2: nếu trạng thái đầu là 500 thì đưa về 200 bằng nhánh `remaining`; kiểm tra lại bằng cách theo dõi từng lượt.

Chương A

Bảng từ khóa và ký hiệu

Cột "Nguồn": **G** là có trong mã gốc của tác giả; **[ĐX]** là đề xuất của người biên soạn, chờ duyệt.

Từ khóa	Ý nghĩa	Nguồn	Chương
`sta`	Statement Authority, khung (thể xác)	G	1
`iid`	Instruction Identity, định danh lệnh (linh hồn)	G	1
`ed`	chốt câu lệnh / khối	G	1
`imp*`	nạp thư viện, dạng `imp*tên.ed`	G	1
`main`	khung chương trình chính	G	1
`pri`, `ipt`	in; nhập	G	3
`ref*`	kết thúc khung, trả giá trị	G	1, 7
`=>`	nối đầu lệnh với thẻ và thân	**[ĐX]**	1
`cdn`	thẻ điều kiện	G	4, 5
`chck`	thẻ kiểm soát bước (`for`)	G	5
`edt`	thẻ chỉnh sửa giá trị	G	2, 5
`dta`	thẻ dữ liệu (`struct`)	G	8
`unkw`	thẻ "ngoài dữ liệu" (`void`)	G	7
`tf`	thẻ đúng/sai (`bolean`)	G	4
`rtn`	thẻ trả giá trị (`fn`)	**[ĐX]**	7
`bolean`	khối rẽ nhánh	G	4
`for`, `while`	vòng lặp	G	5
`struct`	cấu trúc	G	8
`void`	thủ tục / chốt khép thủ tục	G	7
`fn`	hàm có kết quả / chốt khép hàm	**[ĐX]**	7
`dbl`	họ đôi: mở thân và khép vòng lặp	G	5
`strt`	chốt khép `struct`	G	8
`bln`	chốt khép `bolean`	G	4
`else`, `brk`, `nxt`, `call`	nhánh còn lại; thoát vòng; sang lượt kế; gọi	**[ĐX]**	4, 5, 7
`and`, `or`, `not`	toán tử logic	**[ĐX]**	2, 4
`int`, `float`, `str`	kiểu số nguyên, số thực, chuỗi	G	2
`bool`, `true`, `false`	kiểu và giá trị đúng/sai	G / **[ĐX]**	2, 4
`/s{}`, `/d{}`, `/f{}`	chỗ trống định dạng	G (`{ }` là **[ĐX]**)	3
`/b{}`, `/f3{}`, `//`	chỗ trống bool, 3 chữ số lẻ, dấu `/`	**[ĐX]**	3
`\\ ... \\`	ghi chú khối	G	1
`*/`	ghi chú một dòng (Sd++)	G	10
`'`	ngăn các mục dữ liệu	G	5, 6
`,`	ngăn thẻ đặc tả	G	4, 5
`ssta`, `##`	mở và đóng chương trình Sd++	G	10
`$`, `₫`, `€`, `$$`, `#`	bộ cờ chốt	G	10

Từ khóa	Ý nghĩa	Nguồn	Chương
`define[ ]{ } €`	khung ngoài, đổi tên	G	11
`ids.g`, `ids(x).g`	khai báo và gọi biến toàn cục	G	11
`iptedt.int`, `iptedt.ptr`	khai báo cục bộ dạng phẳng	G	11
`ptr`, `@x`, `*p`	con trỏ, địa chỉ, giải tham chiếu	G	12
`sta.alloc`, `sta.free`	cấp phát, giải phóng Heap	G	12
`ids sb`, `block`, `remaining`	khối chọn lọc	G	13
`transf`, `break`	thoát khỏi	G (`break $` là **[ĐX]**)	13
`defk`, `init`, `func`	cờ chốt tùy biến	G	14

Chương B

Ngữ pháp hình thức (EBNF)

Ký hiệu: `[ ]` tùy chọn, `{ }` lặp không hoặc nhiều lần, | lựa chọn; chữ đặt trong nháy kép là chữ nguyên văn. Đây là bản rút gọn dành cho Sd 2 (Phần I).

Ngữ pháp Sd 2

```
chuong_trinh ::= { import } { don_vi } main
import       ::= "imp*" TEN ".ed"
don_vi       ::= khoi_fn | khoi_void | khoi_struct
main         ::= "sta(main){" { lenh } "iid: ref*" bieu_thuc "ed" "}"

lenh         ::= lenh_don | khoi
lenh_don     ::= "iid:" noi_dung "ed"
noi_dung     ::= khai_bao | gan | in | nhap | goi | tra_ve | nhay
khai_bao     ::= "sta(" KIEU TEN [ "[" SO "]" ] ")"
                 [ "=>" "edt:" "sta(" TEN "=" gia_tri ")" ]
gan          ::= "sta(" nhan "=" bieu_thuc ")" | "sta(" nhan ("++"|"--") ")"
in           ::= "sta(pri)" "=>" bieu_thuc
nhap         ::= "sta(ipt)" "=>" "edt:" "sta(" nhan ")"
goi          ::= "call" TEN "(" [ bieu_thuc { "'" bieu_thuc } ] ")"
tra_ve       ::= "ref*" bieu_thuc
nhay         ::= "brk" | "nxt"
nhan         ::= TEN { "[" bieu_thuc "]" | "." TEN }
gia_tri      ::= bieu_thuc | "[" bieu_thuc { "'" bieu_thuc } "]"

khoi         ::= khoi_bolean | khoi_for | khoi_while
khoi_bolean  ::= "iid: sta(bolean) => cdn,tf:"
                 { "sta(" bieu_thuc ") {" {lenh} "}" }
                 [ "sta(else) {" {lenh} "}" ]  "bln ed"
khoi_for     ::= "iid: sta(for) => cdn,chck:"
                 "dbl sta(" [KIEU] TEN "=" bieu_thuc "'" bieu_thuc "'" buoc ") {"
                 {lenh} "}"  "dbl ed"
khoi_while   ::= "iid: sta(while) => cdn,edt:"
                 "dbl sta(" bieu_thuc ") {" {lenh} "}"  "dbl ed"
khoi_fn      ::= "iid: sta(fn) => rtn:"
                 "sta" TEN "(" [tham_so {"'" tham_so}] ")" "->" KIEU "{"
                 {lenh} "iid: ref*" bieu_thuc "ed" "}"  "fn ed"
khoi_void    ::= "iid: sta(void) => unkw:"
                 "sta" TEN "(" [tham_so {"'" tham_so}] ") {" {lenh} "}"  "void ed"
khoi_struct  ::= "iid: sta(struct) => dta:"
                 "sta" TEN "{" KIEU TEN { "'" KIEU TEN } "}"  "strt ed"
tham_so      ::= KIEU TEN [ "[" SO "]" ]
KIEU         ::= "int" | "float" | "str" | "bool" | TEN_STRUCT
ghi_chu      ::= "\\" ... "\\"       (bo qua; thieu dong -> E101)
```

Bảng thẻ đặc tả và chốt khép

```
Khoi     The dac ta   Khep       Ho
-------  -----------  ---------  ------------------------------------------
for      cdn,chck     dbl ed     Ho doi (hai the)
while    cdn,edt      dbl ed     Ho doi
bolean   cdn,tf       bln ed     Ho logic
struct   dta          strt ed    Ho don (mot the)
void     unkw         void ed    Ho don
fn       rtn          fn ed      Ho ham
main     (khong)      ref* ... }   khep bang lenh tra ve
```

Chương C

Bảng mã lỗi và cảnh báo

Toàn bộ mã dưới đây do người biên soạn đề xuất[ĐX] để thống nhất thông báo; tác giả có thể đổi số hiệu.

Mã	Ý nghĩa	Chương
Từ vựng		
E101	ghi chú khối mở mà không đóng (báo dòng mở)	1
E102	ký tự không hợp lệ	16
E103	chuỗi chưa đóng nháy	16
Cú pháp		
E201	thiếu chốt `ed` (hoặc cờ chốt) ở cuối lệnh	9
E202	khối chưa khép, hoặc khép sai loại	4, 16
E203	thiếu hoặc thừa `}`	9
E204	`main` không kết thúc bằng `ref*`	1, 9
E205	dùng thư viện chưa `imp*`	15
E206	đầu vòng `for` thiếu mục khởi tạo, điều kiện hoặc bước	5
E207	hàm có kiểu trả về nhưng thiếu `ref*` cuối cùng	7
E208	chương trình Sd++ thiếu `ssta` hoặc `##`	10
E209	định nghĩa lại cờ dành riêng hoặc cờ đã niêm phong	14
E210	khối `defk` thiếu `#` niêm phong	14
E211	nạp tệp vòng tròn[ĐX]	15
E213	`:&` đặt bên trong một khối[ĐX]	14
E214	vừa có `main` vừa có lệnh viết thẳng trong `ssta`[ĐX]	14
E215	`imp*` không tìm thấy tệp[ĐX]	15
Ngữ nghĩa		
E301	biến chưa khai báo	2
E302	trùng tên (biến, trường, hàm)	2, 8, 11
E303	sai kiểu, kể cả dùng `=` làm điều kiện	2
E304	gọi hàm sai số lượng hoặc kiểu đối số	7
E305	`ref*` ngoài khung hợp lệ; `brk`, `nxt` ngoài vòng lặp	5, 7
E306	`sta(else)` không nằm cuối khối `bolean`	4
Khi chạy		
E401	chia cho 0	9
E402	Reback vượt giới hạn 65 536 lần	13
E403	nhập sai kiểu dữ liệu	3
E404	chỉ số mảng ngoài khoảng	6
E405	giải tham chiếu con trỏ rỗng	12
E406	`sta.free` con trỏ rỗng hoặc đã giải phóng	12
E407	hết bộ nhớ Heap	12
E408	đọc ghi ngoài vùng Heap đã cấp	12
E409	`$$` đặt ngoài khối `ids sb` (dừng cứng)[ĐX]	13
E500	lỗi nội bộ của bộ thông dịch[ĐX]	16

Mã	Ý nghĩa	Chương
`E501`	vượt giới hạn số bước hoặc độ dài kết quả **[ĐX]**	16
Cảnh báo		
`W201`	vòng `while` không chỉnh biến trong điều kiện	5
`W202`	dùng dạng cũ (`/dtemp`, `break $$`, `transf $ $`, mảng ngăn bằng phẩy)	3, 11, 13
`W203`	nhánh Reback không thấy sửa biến của `sb`	13
`W301`	biến hoặc hàm khai báo mà không dùng	2
`W302`	`sta.alloc` không có `sta.free` tương ứng	12

Chương D

Đối chiếu bản gốc và Sd 2

Phụ lục này liệt kê mọi thay đổi so với mã gốc, kèm lý do. Tác giả có thể chấp nhận từng dòng, hoặc giữ nguyên bản gốc ở dòng nào không đồng ý; phần còn lại của sách không phụ thuộc lẫn nhau, trừ những chỗ ghi rõ.

D.1 Từ bản chụp màn hình sang Sd 2

#	Bản gốc	Vấn đề	Sd 2
1	`==` nối đầu với thẻ và cũng là so sánh	một ký hiệu hai nghĩa	`=>` để nối; `==` chỉ so sánh
2	`sta(int i, ipt) == edt:`	`ipt` vừa như kiểu, vừa là lệnh nhập	`ipt` chỉ là lệnh nhập (chương 3)
3	ghi chú `\\ ... \\`; bản nén dùng `\`	hai dạng, dễ nuốt mã	chỉ `\\ ... \\`; chưa đóng báo `E101` kèm dòng mở
4	`sta(i ' i < 10 ' i++)`	thiếu khởi tạo	`sta(int i = 0 ' i < 10 ' i++)`
5	`double sta(int b = 0) == sta(i < 0)`	khai báo `b` nhưng điều kiện dùng `i`, sai ngay từ đầu	`dbl sta(điều kiện)`, thân phải chỉnh biến điều kiện (`W20⌟1`)
6	`double` mở vòng, `dbl` khép	dễ nhầm với kiểu số thực	thống nhất `dbl` ở cả hai chỗ
7	`arr[int a(10) ' ... float a(1.4)]`	trộn mảng và cấu trúc, trùng tên `a`	`sta Tên { kiểu tên ' ... }`; tên trường duy nhất
8	`sta nondta{ int c = 2 ' iid: ... ' float d = 32.2 }`	trộn khai báo và lệnh bằng `'`, khối không tên	thủ tục có tên: `sta ten() { ... }`, gọi bằng `call`
9	`sta chcktf (true, fa⌟lse) == sta(a = 0)`	dùng phép gán làm điều kiện	khối `bolean` với `sta(a == 0⌟)` và `sta(else)`
10	`sta() diem { diem: ... }`	tên lặp, không có tham số hay kiểu	`iid: sta(fn) => rtn: … fn ed`
11	`ref* 0 ed:`	dấu `:` thừa	`iid: ref* 0 ed`; `main` bắt buộc có
12	`imp*ioprt.ed` và `impio⌟prt.ed`	hai dạng viết	chỉ `imp*tên.ed`
13	`,` và `'` dùng chung	không phân vai	`,` ngăn thẻ; `'` ngăn dữ liệu
14	`/dtemp`	nhập nhằng ranh giới tên	`/d{temp}`
15	mỗi khối tự chọn cách đóng	kiểu đóng không tính được từ khối	quy tắc khép kín: `tên-khép ed` theo họ (Phụ lục B)

D.2 Từ đặc tả Sd++ cũ sang Sd++ 2

Bản gốc	Vấn đề	Sd++ 2
`break $$` và `transf $ $`	`$$` vừa là Reback vừa là "kết thúc"	`$$` chỉ là Reback; thoát bằng `tra⌋nsf $` và `break $`
Reback không có giới hạn	có thể lặp vô hạn	`W203` lúc biên dịch, `E402` ở 65 536 lần
`sta.arr.g: so = [10, 20] ₫`	dấu phẩy ngăn phần tử	`[10 ' 20]`; dạng cũ cảnh báo `W⌋202`
`/dtemp`, `/sids(x).g`	nhập nhằng	`/d{temp}`, `/s{ids(x).g}`
Hai bộ dấu chốt, Sd++ và Sd tách rời	học hai lần	`ed` là `$` ở cấp câu lệnh; Sd là tập con của Sd++
`sta.free(p)` không nói gì về `p`	dùng lại con trỏ	`p` thành rỗng; `E405` và `E406`
Không rõ ô nhớ lớn bao nhiêu	không tính được `alloc(n)`	một ô nhớ 8 byte
Thiếu `##` không có lỗi riêng	có thể không dọn tài nguyên	`E208`

Chương E

Bảng tra nhanh

Sd 2: mọi dạng câu lệnh trên một trang

```
CHUONG TRINH
  imp*ioprt.ed
  sta(main){ ... iid: ref* 0 ed }

LENH DON (moi lenh ket thuc bang ed)
  iid: sta(int a) ed                              khai bao
  iid: sta(int a) => edt: sta(a = 5) ed           khai bao + gia tri dau
  iid: sta(a = a + 1) ed                          gan (rut gon);  a++  a--
  iid: sta(int m[3]) => edt: sta(m = [1 ' 2 ' 3]) ed       mang
  iid: sta(pri) => "a=/d{a} s=/s{t} f=/f{x}" ed            in
  iid: sta(ipt) => edt: sta(a) ed                          nhap
  iid: call ten(1 ' 2) ed                                  goi
  iid: brk ed     iid: nxt ed                              thoat / bo luot

RE NHANH
  iid: sta(bolean) => cdn,tf:
      sta(dk1) { ... }
      sta(dk2) { ... }
      sta(else) { ... }
  bln ed

VONG LAP
  iid: sta(for) => cdn,chck:
      dbl sta(int i = 0 ' i < n ' i++) { ... }
  dbl ed
  iid: sta(while) => cdn,edt:
      dbl sta(dieu_kien) { ...chinh bien trong dieu kien... }
  dbl ed

HAM / THU TUC / CAU TRUC
  iid: sta(fn) => rtn:
      sta ten(int a ' int b) -> int { ... iid: ref* a + b ed }
  fn ed
  iid: sta(void) => unkw:
      sta ten(str s) { ... }
  void ed
  iid: sta(struct) => dta:
      sta Ten { str a ' int b }
  strt ed

TOAN TU  (cao->thap)  ( ) [ ] .  |  not -  |  * / %  |  + -  |  < <= > >=
                      |  == !=  |  and  |  or
KIEU      int float str bool         MAC DINH  0  0.0  ""  false
GHI CHU   \\ nhieu dong \\          NGAN CACH  "," giua the   "'" giua du lieu
```

Sd++: bộ cờ và khối chính

```
ssta { ... }  ##        hop bao ve: mo / dong tat ca
$    chot lenh trong khung          đ   chot bien toan cuc tran
€    chot khung define              #   niem phong defk ; diem neo
$$   Reback (trong ids sb) ; ngoai sb = dung khan cap
ids.g: ten = gia_tri đ              bien toan cuc ;  ids(ten).g  goi lai
iptedt.int a = 5 $    iptedt.ptr p = @a $    *p = 7 $
sta.alloc(n)          sta.free(p)
ids sb(bien) {
    block dk:   ...  transf $       hoan tat, thoat khoi
    block dk:   ...  sua bien  $$   Reback: xet lai tu dau
    remaining:  ...  break $        thoat ngay
} $
defk<ids.end: text "¥"> { init ids.edt (...)  func * end.sta * end.iid # }
define[ ids.edt: text = "main", pri.edt: text = "print" ]{
    main: { ... } $
} €
```

Chương F

Đáp án bài tập

Hãy tự làm trước khi xem đáp án. Với bài có nhiều cách giải, đáp án chỉ nêu một cách; kết quả ghi theo đặc tả (xem trang thông tin tài liệu).

Chương 1

1.1 Đáp án 1.1

```
imp*ioprt.ed

sta(main){
    iid: sta(pri) => "Nguyen Van A" ed
    iid: sta(pri) => "Lop 10A1" ed
    iid: ref* 0 ed
}
```

1.2 Thêm ở đầu tệp, ví dụ: `\\ Ngay viet: 04/11/2026. Muc dich: bai tap chuong 1. \\`

1.3 Dòng một thiếu chốt `ed`; dòng hai thiếu dấu hai chấm sau `iid`; dòng ba đúng.

1.4 Thiếu `imp*ioprt.ed` (lỗi `E205`) và thiếu `iid: ref* 0 ed` trước `}` (lỗi `E204`).

Chương 2

2.1 `3`, `2`, `3.4`, `14`, `20`.

2.2 Đáp án 2.2

```
imp*ioprt.ed

sta(main){
    iid: sta(int a) => edt: sta(a = 12) ed
    iid: sta(int b) => edt: sta(b = 5) ed
    iid: sta(pri) => "tong = /d{a + b}" ed
    iid: sta(pri) => "hieu = /d{a - b}" ed
    iid: sta(pri) => "tich = /d{a * b}" ed
    iid: sta(pri) => "thuong = /d{a / b}, du = /d{a % b}" ed
    iid: ref* 0 ed
}
```

Kết quả: `tong = 17`, `hieu = 7`, `tich = 60`, `thuong = 2, du = 2`.

2.3 Đáp án 2.3

```
imp*ioprt.ed

sta(main){
    iid: sta(float r) => edt: sta(r = 2.5) ed
    iid: sta(float pi) => edt: sta(pi = 3.14159) ed
    iid: sta(pri) => "chu vi = /f{2.0 * pi * r}" ed
    iid: sta(pri) => "dien tich = /f{pi * r * r}" ed
    iid: ref* 0 ed
}
```

Kết quả: `chu vi = 15.71`, `dien tich = 19.63`.

2.4 `sta(a = 0)` là phép gán: nó đổi `a` thành `0` thay vì hỏi `a` có bằng `0`. Sd 2 báo `E303` để tránh nhầm lẫn này.

Chương 3

3.1 Đáp án 3.1

```
imp*ioprt.ed

sta(main){
    iid: sta(str ten) ed
    iid: sta(int tuoi) ed
    iid: sta(pri) => "Ten:" ed
    iid: sta(ipt) => edt: sta(ten) ed
    iid: sta(pri) => "Tuoi:" ed
    iid: sta(ipt) => edt: sta(tuoi) ed
    iid: sta(pri) => "Xin chao /s{ten}, nam sau /d{tuoi + 1} tuoi" ed
    iid: ref* 0 ed
}
```

3.2 Đáp án 3.2

```
imp*ioprt.ed

sta(main){
    iid: sta(float dai) ed
    iid: sta(float rong) ed
    iid: sta(ipt) => edt: sta(dai) ed
    iid: sta(ipt) => edt: sta(rong) ed
    iid: sta(pri) => "Chu vi = /f{2.0 * (dai + rong)}" ed
    iid: sta(pri) => "Dien tich = /f{dai * rong}" ed
    iid: ref* 0 ed
}
```

3.3 Đáp án 3.3

```
imp*ioprt.ed

sta(main){
    iid: sta(int n) ed
    iid: sta(ipt) => edt: sta(n) ed
    iid: sta(pri) => "Tram: /d{n / 100}" ed
    iid: sta(pri) => "Chuc: /d{n / 10 % 10}" ed
    iid: sta(pri) => "Don vi: /d{n % 10}" ed
    iid: ref* 0 ed
}
```

Vì `/` và `%` cùng bậc nên `n / 10 % 10` tính từ trái sang là `(n / 10) % 10`.

3.4 `iid: sta(pri) => "Ti le: 3//4" ed`

Chương 4

4.1 Đáp án 4.1

```
imp*ioprt.ed

sta(main){
    iid: sta(int n) ed
    iid: sta(ipt) => edt: sta(n) ed
    iid: sta(bolean) => cdn,tf:
        sta(n % 2 == 0) {
            iid: sta(pri) => "chan" ed
        }
        sta(else) {
            iid: sta(pri) => "le" ed
        }
    bln ed
    iid: ref* 0 ed
}
```

4.2 Đáp án 4.2

```
imp*ioprt.ed

sta(main){
    iid: sta(int a) ed
    iid: sta(int b) ed
    iid: sta(int c) ed
    iid: sta(ipt) => edt: sta(a) ed
    iid: sta(ipt) => edt: sta(b) ed
    iid: sta(ipt) => edt: sta(c) ed
    iid: sta(int m) => edt: sta(m = a) ed
    iid: sta(bolean) => cdn,tf:
        sta(b > m) {
            iid: sta(m = b) ed
        }
    bln ed
    iid: sta(bolean) => cdn,tf:
        sta(c > m) {
            iid: sta(m = c) ed
        }
    bln ed
    iid: sta(pri) => "Lon nhat: /d{m}" ed
    iid: ref* 0 ed
}
```

4.3 Đáp án 4.3 (phần lõi)

```
iid: sta(bolean) => cdn,tf:
    sta(n % 400 == 0 or (n % 4 == 0 and n % 100 != 0)) {
        iid: sta(pri) => "nhuan" ed
    }
    sta(else) {
        iid: sta(pri) => "khong nhuan" ed
    }
bln ed
```

4.4 Đáp án 4.4

```
imp*ioprt.ed

sta(main){
    iid: sta(float a) ed
    iid: sta(float b) ed
```

```
    iid: sta(ipt) => edt: sta(a) ed
    iid: sta(ipt) => edt: sta(b) ed
    iid: sta(bolean) => cdn,tf:
        sta(a != 0.0) {
            iid: sta(pri) => "x = /f{-b / a}" ed
        }
        sta(b == 0.0) {
            iid: sta(pri) => "vo so nghiem" ed
        }
        sta(else) {
            iid: sta(pri) => "vo nghiem" ed
        }
    bln ed
    iid: ref* 0 ed
}
```

Chương 5

5.1 Đáp án 5.1

```
sta(main){
    iid: sta(for) => cdn,chck:
        dbl sta(int i = 2 ' i <= 20 ' i = i + 2) {
            iid: sta(pri) => "/d{i}" ed
        }
    dbl ed
    iid: ref* 0 ed
}
```

5.2 Đáp án 5.2

```
imp*ioprt.ed

sta(main){
    iid: sta(int n) ed
    iid: sta(ipt) => edt: sta(n) ed
    iid: sta(int gt) => edt: sta(gt = 1) ed
    iid: sta(for) => cdn,chck:
        dbl sta(int i = 2 ' i <= n ' i++) {
            iid: sta(gt = gt * i) ed
        }
    dbl ed
    iid: sta(pri) => "/d{n}! = /d{gt}" ed
    iid: ref* 0 ed
}
```

5.3 Đáp án 5.3 (với n $>$ 0)

```
imp*ioprt.ed

sta(main){
    iid: sta(int n) ed
    iid: sta(ipt) => edt: sta(n) ed
    iid: sta(int dem) => edt: sta(dem = 0) ed
    iid: sta(while) => cdn,edt:
        dbl sta(n > 0) {
            iid: sta(dem++) ed
            iid: sta(n = n / 10) ed
```

```
        }
    dbl ed
    iid: sta(pri) => "So chu so: /d{dem}" ed
    iid: ref* 0 ed
}
```

5.4 Đáp án 5.4

```
imp*ioprt.ed
imp*str.ed

sta(main){
    iid: sta(str dong) ed
    iid: sta(for) => cdn,chck:
        dbl sta(int k = 1 ' k <= 4 ' k++) {
            iid: sta(dong = str.concat(dong ' "*")) ed
            iid: sta(pri) => "/s{dong}" ed
        }
    dbl ed
    iid: ref* 0 ed
}
```

5.5 Thân vòng không làm `n` tiến về `0`, nên `n != 0` mãi đúng (`W201`). Thêm `iid: sta(n = n - 1) ed` vào thân.

Chương 6

6.1 Đáp án 6.1

```
imp*ioprt.ed

sta(main){
    iid: sta(int a[5]) => edt: sta(a = [3 ' 8 ' 2 ' 6 ' 1]) ed
    iid: sta(int s) => edt: sta(s = 0) ed
    iid: sta(for) => cdn,chck:
        dbl sta(int i = 0 ' i < 5 ' i++) {
            iid: sta(s = s + a[i]) ed
        }
    dbl ed
    iid: sta(pri) => "Tong = /d{s}" ed
    iid: sta(pri) => "TB = /f{s / 5.0}" ed
    iid: ref* 0 ed
}
```

Kết quả: `Tong = 20`, `TB = 4.00`.

6.2 Đáp án 6.2

```
imp*ioprt.ed

sta(main){
    iid: sta(int a[5]) => edt: sta(a = [3 ' 8 ' 2 ' 6 ' 1]) ed
    iid: sta(int dem) => edt: sta(dem = 0) ed
    iid: sta(for) => cdn,chck:
        dbl sta(int i = 0 ' i < 5 ' i++) {
            iid: sta(bolean) => cdn,tf:
                sta(a[i] % 2 == 0) {
                    iid: sta(dem++) ed
                }
```

```
            bln ed
        }
    dbl ed
    iid: sta(pri) => "So phan tu chan: /d{dem}" ed
    iid: ref* 0 ed
}
```

Kết quả: `So phan tu chan: 3`.

6.3 Đáp án 6.3 (phần lõi)

```
iid: sta(for) => cdn,chck:
    dbl sta(int i = 0 ' i < 2 ' i++) {
        iid: sta(int t) => edt: sta(t = a[i]) ed
        iid: sta(a[i] = a[4 - i]) ed
        iid: sta(a[4 - i] = t) ed
    }
dbl ed
```

6.4 Xem đáp án 5.4.

6.5 Đáp án 6.5

```
imp*ioprt.ed
imp*str.ed

sta(main){
    iid: sta(str s) ed
    iid: sta(ipt) => edt: sta(s) ed
    iid: sta(bolean) => cdn,tf:
        sta(str.len(s) == 0) {
            iid: sta(pri) => "rong" ed
        }
        sta(else) {
            iid: sta(pri) => "Do dai: /d{str.len(s)}" ed
        }
    bln ed
    iid: ref* 0 ed
}
```

Chương 7

7.1 Đáp án 7.1

```
imp*ioprt.ed

iid: sta(fn) => rtn:
    sta lonnhat(int a ' int b) -> int {
        iid: sta(int m) => edt: sta(m = a) ed
        iid: sta(bolean) => cdn,tf:
            sta(b > a) {
                iid: sta(m = b) ed
            }
        bln ed
        iid: ref* m ed
    }
fn ed

sta(main){
```

```
    iid: sta(pri) => "Lon nhat: /d{lonnhat(lonnhat(4 ' 9) ' 6)}" ed
    iid: ref* 0 ed
}
```

7.2 Đáp án 7.2

```
imp*ioprt.ed

iid: sta(fn) => rtn:
    sta lachinhphuong(int n) -> bool {
        iid: sta(bool kq) => edt: sta(kq = false) ed
        iid: sta(for) => cdn,chck:
            dbl sta(int k = 1 ' k * k <= n ' k++) {
                iid: sta(bolean) => cdn,tf:
                    sta(k * k == n) {
                        iid: sta(kq = true) ed
                    }
                bln ed
            }
        dbl ed
        iid: ref* kq ed
    }
fn ed

sta(main){
    iid: sta(for) => cdn,chck:
        dbl sta(int i = 1 ' i <= 50 ' i++) {
            iid: sta(bolean) => cdn,tf:
                sta(lachinhphuong(i)) {
                    iid: sta(pri) => "/d{i}" ed
                }
            bln ed
        }
    dbl ed
    iid: ref* 0 ed
}
```

Kết quả: `1 4 9 16 25 36 49`, mỗi số một dòng.

7.3 Đáp án 7.3

```
imp*ioprt.ed
imp*str.ed

iid: sta(void) => unkw:
    sta invach(int n) {
        iid: sta(str s) ed
        iid: sta(for) => cdn,chck:
            dbl sta(int i = 0 ' i < n ' i++) {
                iid: sta(s = str.concat(s ' "-")) ed
            }
        dbl ed
        iid: sta(pri) => "/s{s}" ed
    }
void ed

sta(main){
    iid: call invach(10) ed
    iid: ref* 0 ed
}
```

7.4 Đáp án 7.4

```
imp*ioprt.ed

iid: sta(fn) => rtn:
    sta fib(int n) -> int {
        iid: sta(bolean) => cdn,tf:
            sta(n <= 1) {
                iid: ref* n ed
            }
        bln ed
        iid: ref* fib(n - 1) + fib(n - 2) ed
    }
fn ed

sta(main){
    iid: sta(pri) => "fib(10) = /d{fib(10)}" ed
    iid: ref* 0 ed
}
```

Kết quả: `fib(10) = 55`.

Chương 8

8.1 Đáp án 8.1

```
imp*ioprt.ed

iid: sta(struct) => dta:
    sta PhanSo { int tu ' int mau }
strt ed

iid: sta(fn) => rtn:
    sta cong(PhanSo a ' PhanSo b) -> PhanSo {
        iid: sta(PhanSo r) ed
        iid: sta(r.tu = a.tu * b.mau + b.tu * a.mau) ed
        iid: sta(r.mau = a.mau * b.mau) ed
        iid: ref* r ed
    }
fn ed

sta(main){
    iid: sta(PhanSo x) ed
    iid: sta(PhanSo y) ed
    iid: sta(x.tu = 1) ed
    iid: sta(x.mau = 2) ed
    iid: sta(y.tu = 1) ed
    iid: sta(y.mau = 3) ed
    iid: sta(PhanSo z) => edt: sta(z = cong(x ' y)) ed
    iid: sta(pri) => "/d{z.tu} tren /d{z.mau}" ed
    iid: ref* 0 ed
}
```

Kết quả: `5 tren 6`.

8.2 Xem Dự án 1 ở chương 17, bỏ phần xếp loại và in điểm trung bình.

8.3 Trường `hang` xuất hiện hai lần: lỗi `E302` (tên trường phải duy nhất).

8.4 `SinhVien` là tên *kiểu*, không phải biến. Phải khai báo biến `sta(SinhVien sv)` rồi gán `sv.ten`.

Chương 9

9.1 `E201` (thiếu chốt).

9.2 Đáp án 9.2

```
imp*ioprt.ed

sta(main){
    iid: sta(int a) ed
    iid: sta(int b) ed
    iid: sta(ipt) => edt: sta(a) ed
    iid: sta(ipt) => edt: sta(b) ed
    iid: sta(bolean) => cdn,tf:
        sta(b == 0) {
            iid: sta(pri) => "chia cho 0" ed
            iid: ref* 3 ed
        }
    bln ed
    iid: sta(pri) => "Thuong = /d{a / b}" ed
    iid: ref* 0 ed
}
```

9.3 Lỗi `E404`: mảng 5 phần tử có chỉ số từ 0 đến 4.

9.4 Đáp án 9.4

```
imp*ioprt.ed

sta(main){
    iid: sta(int n) => edt: sta(n = 10) ed
    iid: sta(while) => cdn,edt:
        dbl sta(n != 0) {
            iid: sta(pri) => "/d{n}" ed
            iid: sta(n = n - 1) ed
        }
    dbl ed
    iid: ref* 0 ed
}
```

Chương 10 đến 12

10.1 Đáp án 10.1 (khung main)

```
main: {
    ids.pri: "Dong mot" $
    ids.pri: "Dong hai" $
    ref*0 $:
} $
```

10.2 (a) đ̲ (b) `$` (c) `€`.

10.3 Lỗi `E208`; không có nắp đóng thì việc dọn sạch Stack, thanh ghi và Heap không được bảo đảm.

11.1 Đáp án 11.1

```
ids.g: ten = "An" đ̲
ids.g: tuoi = 20 đ̲
ids.pri: "/s{ids(ten).g}, /d{ids(tuoi).g} tuoi" $
```

11.2 Biến toàn cục nằm ngoài mọi khung nên không có phạm vi để chốt: dùng cờ riêng đ. Biến trong khung chốt bằng `$` như mọi câu lệnh.

11.3 Đáp án 11.3

```
iid: sta(int a) => edt: sta(a = 5) ed
iid: sta(pri) => "hi" ed
```

12.1 Đáp án 12.1 (khung main)

```
main: {
    iptedt.int x = 3 $
    iptedt.int y = 8 $
    iptedt.ptr px = @x $
    iptedt.ptr py = @y $
    iptedt.int t = *px $
    *px = *py $
    *py = t $
    ids.pri: "x=/d{x} y=/d{y}" $
    ref*0 $:
} $
```

Kết quả: `x=8 y=3`.

12.2 Đáp án 12.2 (khung main)

```
main: {
    iptedt.ptr h = sta.alloc(32) $
    *h = 10 $
    *(h + 1) = 20 $
    *(h + 2) = 30 $
    *(h + 3) = 40 $
    ids.pri: "Tong = /d{*h + *(h + 1) + *(h + 2) + *(h + 3)}" $
    sta.free(h) $
    ref*0 $:
} $
```

Kết quả: `Tong = 100`.

12.3 Sau `sta.free(h)`, `h` thành rỗng; `*h = 1` gây `E405`.

Chương 13 đến 17

13.1 Đáp án 13.1 (khung main)

```
main: {
    iptedt.int score = -15 $
    ids sb(score) {
        block score > 0:
            ids.pri: "Duyet diem thanh cong: /d{score}" $
            transf $
        block score <= 0:
            ids.pri: "Diem /d{score}, cong 30" $
            score = score + 30 $
            $$
    } $
    ref*0 $:
} $
```

Kết quả: `Diem -15, cong 30` rồi `Duyet diem thanh cong: 15`.

13.2 Đáp án 13.2

```
imp*ioprt.ed

sta(main){
    iid: sta(int score) => edt: sta(score = -15) ed
    iid: sta(while) => cdn,edt:
        dbl sta(score <= 0) {
            iid: sta(pri) => "Diem /d{score}, cong 30" ed
            iid: sta(score = score + 30) ed
        }
    dbl ed
    iid: sta(pri) => "Duyet diem thanh cong: /d{score}" ed
    iid: ref* 0 ed
}
```

13.3 Biến không đổi thì điều kiện không đổi, Reback lặp mãi. Trình biên dịch cảnh báo `W203`; nếu vẫn chạy thì dừng ở `E402`.

14.1 Đáp án 14.1

```
defk<ids.end: text "¤"> {
    init ids.edt ("¤" endlogic == "$"+"€"+"$$")
    func * end.sta * end.iid #
}
```

14.2 `$` là cờ dành riêng; định nghĩa lại nó làm đổi nghĩa của mọi chương trình (lỗi `E209`).

14.3 Ví dụ F.1. Đáp án 14.3

```
ssta {
    init.endcomand: ":&" $
    ids.pri: "Dong 1"
    ids.pri: "Dong 2"
    ids.pri: "Dong 3"
    :&
}
##
```

14.4 Ví dụ F.2. Đáp án 14.4

```
ssta {
    init.endcomand: ":&" $
    ids.pri: "Vung tu chot"
    :&
    ids.pri: "Vung chot tay" $
}
##
```

15.1 Đáp án 15.1

```
imp*ioprt.ed
imp*math.ed

sta(main){
    iid: sta(float a) ed
    iid: sta(float b) ed
```

```
    iid: sta(ipt) => edt: sta(a) ed
    iid: sta(ipt) => edt: sta(b) ed
    iid: sta(float c) => edt: sta(c = math.sqrt(a * a + b * b)) ed
    iid: sta(pri) => "Canh huyen = /f{c}" ed
    iid: ref* 0 ed
}
```

15.2 Giống ví dụ ghi và đọc tệp ở chương 15, đổi nội dung chuỗi.

16.1 `iid : sta ( int a ) => edt : sta ( a = 5 ) ed`.

16.2 Ngăn xếp còn hai phần tử `bln`; báo `E202` (chưa khép) cho mỗi khối.

17.1 Đáp án 17.1 (thêm vào trước ref* của main)

```
iid: sta(int vt) => edt: sta(vt = 0) ed
iid: sta(for) => cdn,chck:
    dbl sta(int i = 1 ' i < 3 ' i++) {
        iid: sta(bolean) => cdn,tf:
            sta(ds[i].diem > ds[vt].diem) {
                iid: sta(vt = i) ed
            }
        bln ed
    }
dbl ed
iid: sta(pri) => "Cao nhat: /s{ds[vt].ten}" ed
```

17.2 Lượt 1: không nhánh `block` nào đúng (500 khác 200 và không âm) nên chạy `remaining`, đặt `200`, Reback. Lượt 2: `status == 200`, in ONLINE và `transf` thoát khối.

Chương G

Sd IDE, bộ thông dịch và kết quả kiểm chứng

Kèm sách có ba thứ do người biên soạn viết để kiểm chứng và để bạn thực hành. Tất cả là **công cụ tham chiếu**[ĐX], không phải trình biên dịch chính thức.

- **Sd IDE**: một tệp HTML chạy ngoại tuyến trong trình duyệt. Có soạn thảo nhiều tệp, chạy Sd và Sd++, nhập liệu tương tác, gỡ lỗi từng bước (điểm dừng, xem biến, ngăn xếp gọi), bảng "Vấn đề" hiện lỗi và cảnh báo, ví dụ có sẵn, mở thư mục dự án và đánh dấu tệp chính bằng dấu sao.
- **sd.js**: bản dòng lệnh chạy bằng Node.js, `node sd.js bai.sd`; đọc dữ liệu nhập từ bàn phím hoặc ống dẫn.
- **sdi.py**: bộ thông dịch Python ban đầu cho Sd 2, dùng để đối chiếu chéo.

Bộ thông dịch của IDE chạy được cả Sd lẫn Sd++: con trỏ, Heap, `ids sb` với Reback, `defk`, biến toàn cục, vùng tự chốt `:&` và dự án nhiều tệp.

Cách dùng dòng lệnh

```
node sd.js chuong_trinh.sd            # nhap du lieu tu ban phim
node sd.js chuong_trinh.sd < vao.txt  # nhap du lieu tu tep
python3 sdi.py chuong_trinh.sd        # ban Python cho Sd

Ma thoat = gia tri cua ref* ; loi khi chay -> [LOI Exxx @dong N].
```

Kết quả kiểm chứng

Chương trình Sd đầy đủ đã chạy, kết quả IDE trùng `sdi.py`	53 / 53
Chạy thường và chạy gỡ lỗi cho kết quả giống nhau	53 / 53
Chương trình Sd++ đầy đủ trong sách đã chạy không lỗi	11
Đoạn mã chỉ là mảnh (không chạy riêng)	7

Các kết quả Sd++ in trong sách đã được so với kết quả chạy thật, và các lỗi `E401`–`E409` đã được thử bằng chương trình cố ý sai.

Giới hạn

- Giới hạn an toàn: 4 triệu bước, 400 000 ký tự kết quả, độ sâu gọi 800, Heap 2 triệu ô; vượt thì báo `E501` hoặc lỗi tương ứng.
- Mô hình `:&` chạy theo hai vùng (xem chương 14, "Điều chưa chốt").
- Bản đóng gói Electron chưa được thử.
- Đây là thông dịch tham chiếu để xác nhận ngữ nghĩa, không phải trình biên dịch mã máy.

Chương H

Thuật ngữ

Biến ô nhớ có tên và kiểu dữ liệu.

Ranh giới tự chốt dòng `:&`; mọi lệnh phía trên nó được thêm kiểu kết ẩn.

Tệp chính tệp có `main` được chạy trong một dự án nhiều tệp.

Chốt (`ed`) dấu kết thúc một câu lệnh hoặc khối.

Cờ chốt ký hiệu kết thúc có vai trò riêng trong Sd++: `$`, `₫`, `€`, `$$`, `#`, `##`.

Đặc tả thực thi (thẻ) từ như `cdn`, `chck`, `edt`, `dta`, `unkw`, `tf`, `rtn` cho biết khối làm việc gì.

Điểm lỗi giá trị trả về cho biết chương trình kết thúc không bình thường (`ref* N`).

Địa chỉ vị trí của một ô nhớ; lấy bằng `@x`.

Heap vùng bộ nhớ xin thêm khi chạy bằng `sta.alloc`.

Họ khối nhóm khối cùng cách khép: họ đôi (`dbl`), họ đơn (`strt`, `void`), họ logic (`bln`), họ hàm (`fn`).

Khép kín quy tắc: mỗi khối mở phải được khép bằng đúng `tên-khép ed` của họ nó.

Lexer, parser bộ phân tích từ vựng; bộ phân tích cú pháp.

Con trỏ biến chứa địa chỉ của một ô nhớ khác.

Reback hành vi quay lại đầu khối `ids sb` để xét lại điều kiện, đặt bằng `$$`.

Token đơn vị nhỏ nhất của mã nguồn sau khi cắt từ vựng.

[ĐX] ký hiệu cho phần đề xuất của người biên soạn, chờ tác giả duyệt.